B.M.S COLLEGE OF ENGINEERING

(Autonomous Institute Affiliated to VTU)
Bull Temple Road, Basavanagudi, Bangalore – 560019

A Project Report on

SmartSME: An AI-Assisted, Event-Driven Business Management Platform for Small and Medium Enterprises

Submitted in partial fulfillment of the requirements for the award of degree of

BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND BUSINESS SYSTEMS

Submitted by

Kousthubha L Rao (1BM23CB026)
Prabhanjan (1BM23CB035)
Pranav Kiran Kumar (1BM23CB036)
Vishwas Kumar S S (1BM23CB063)

Under the guidance of
Dr. Gururaja H S
Associate Professor

Department of Computer Science and Business Systems
B.M.S College of Engineering, Bull Temple Road, Basavanagudi, Bangalore – 560019
2026-2027

B.M.S COLLEGE OF ENGINEERING

(Autonomous Institute Affiliated to VTU)

Bull Temple Road, Basavanagudi, Bangalore – 560019

Department of Computer Science and Business Systems

C E R T I F I C A T E

This is to certify that the Project work entitled "**SmartSME: An AI-Assisted, Event-Driven Business Management Platform for Small and Medium Enterprises**" is a bona-fide work carried out by **Kousthubha L Rao (1BM23CB026), Prabhanjan (1BM23CB035), Pranav Kiran Kumar (1BM23CB036), Vishwas Kumar S S (1BM23CB063)** in partial fulfillment for the award of degree of Bachelor of Engineering in **Computer Science and Business Systems** from **Visvesvaraya Technological University, Belgaum** during the year **2026-2027**. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report deposited in the department library. The Project has been approved as it satisfies the academic requirements in respect of Project work prescribed for the Bachelor of Engineering Degree.

Dr. Gururaja H S Associate Professor & Guide, Dept. of CSBS, BMSCE, Bangalore	**Dr. R Ashok Kumar** Professor & Head, Dept. of CSBS, BMSCE, Bangalore	**Dr. Bheemsha Arya** Professor and Principal, BMSCE, Bangalore

Examiners:

Name of the Examiner	**Signature of the Examiner**
1.	
2.	

DECLARATION BY STUDENTS

We, **Kousthubha L Rao (1BM23CB026), Prabhanjan (1BM23CB035), Pranav Kiran Kumar (1BM23CB036), Vishwas Kumar S S (1BM23CB063)**, students of the Department of Computer Science and Business Systems, B.M.S College of Engineering, Bangalore, hereby declare that the project work entitled "**SmartSME: An AI-Assisted, Event-Driven Business Management Platform for Small and Medium Enterprises**" has been carried out by us under the guidance of **Dr. Gururaja H S**, Associate Professor, Department of Computer Science and Business Systems, B.M.S College of Engineering, and submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science and Business Systems during the academic year 2026-2027.

We further declare that the work reported in this report has not been submitted, either in part or in full, for the award of any other degree or diploma in this or any other institution.

Place: Bangalore

Date:

Name	**USN**	**Signature**
Kousthubha L Rao	1BM23CB026	
Prabhanjan	1BM23CB035	
Pranav Kiran Kumar	1BM23CB036	
Vishwas Kumar S S	1BM23CB063	

ACKNOWLEDGEMENT

Many people helped us carry this project through, and we would like to thank them here. We are grateful to **Dr. Bheemsha Arya**, Principal, B.M.S College of Engineering, for providing the facilities and an environment that made this work possible.

We thank **Dr. R Ashok Kumar**, Professor and Head, Department of Computer Science and Business Systems, for his encouragement and support throughout the project.

We express our sincere gratitude to our guide, **Dr. Gururaja H S**, Associate Professor, Department of Computer Science and Business Systems, for his valuable guidance, constructive suggestions and constant encouragement at every stage of this project.

We also thank the project coordinators and all faculty and staff of the department for their help, and the shop owners and friends who shared how orders are taken in small businesses.

Finally, we thank our parents and friends for their support and patience during the course of this work.

Kousthubha L Rao (1BM23CB026)
Prabhanjan (1BM23CB035)
Pranav Kiran Kumar (1BM23CB036)
Vishwas Kumar S S (1BM23CB063)

ABSTRACT

SmartSME is a business management platform for small and medium enterprises that still take orders through chat messages, phone calls and handwritten slips. Its Smart Input engine converts typed notes in English, Hindi or Kannada, and photographs of order slips, into draft sales, purchases and expenses that are recorded only after a person confirms them. Each confirmed record is committed with its event through a PostgreSQL outbox, from which a rule engine updates stock, balances and alerts. Typed notes were fully correct for 87.1% of inputs offline and 100.0% with a language model, and all twelve slips were read correctly.

Keywords: small and medium enterprises, event-driven architecture, transactional outbox, information extraction, code-mixed text, OCR, business-health score

TABLE OF CONTENTS

Declaration by Students i
Acknowledgement ii
Abstract iii
Table of Contents iv
List of Figures viii
List of Tables x
List of Abbreviations and Acronyms xi
List of Symbols / Notations xii

CHAPTER 1: INTRODUCTION 1
1.1 Background and Context of the Project 1
1.2 Project Domain Overview 2
1.3 Motivation 2
1.4 Problem Statement 2
1.5 Aim and Objectives 3
1.6 Scope of the Project 3
1.7 Significance and Expected Contribution 4
1.8 Industry and Societal Relevance 4
1.9 Sustainable Development Goals (SDGs) Addressed 5
1.9.1 Identification of Relevant SDGs 5
1.9.2 Selected SDG Targets 5
1.9.3 Alignment of Project Objectives with SDGs 5
1.9.4 Expected Social, Economic and Environmental Impact 5
1.10 Constraints and Limitations 6
1.11 Organization of the Report 6
CHAPTER 2: LITERATURE REVIEW AND RELATED WORK 7
2.1 Introduction 7
2.2 Review of Existing Systems / Research Studies 7
2.2.1 ERP and Accounting Software for SMEs 7
2.2.2 Event-Driven Architectures and Message Brokers 7
2.2.3 Event Sourcing and the Transactional Outbox 7
2.2.4 Rule-Based Automation 8
2.2.5 Document Reading: OCR and Document-Understanding Models 8
2.2.6 Large Language and Multimodal Models 8
2.2.7 Code-Mixed Language Processing and Transliteration 8
2.2.8 Measuring Business Health 8
2.3 Comparative Analysis of Existing Approaches 9
2.4 Identified Research / Application Gap 10
2.5 Summary of the Literature Review 10

CHAPTER 3: REQUIREMENTS ANALYSIS AND PROJECT PLANNING 11
3.1 Introduction 11
3.2 Stakeholder Identification and User Needs 11
3.3 Problem / Opportunity Analysis from User and Business Perspectives 12
3.4 Functional Requirements 13
3.5 Non-Functional Requirements 13
3.6 Hardware and Software Requirements 14
3.7 Feasibility Analysis 14
3.7.1 Technical Feasibility 14
3.7.2 Economic / Cost Feasibility 14
3.7.3 Operational and Schedule Feasibility 14
3.8 Project Plan, Milestones and Task Allocation 15
3.9 Risk Assessment and Mitigation 16
3.10 Data Privacy, Security, and Compliance Requirements 17
CHAPTER 4: BUSINESS MODEL AND SUSTAINABILITY FRAMEWORK 18
4.1 Introduction to the Proposed Business / Solution 18
4.2 Identification of Target Users and Customer Segments 18
4.3 Customer Problem and Value Proposition 19
4.4 Business Model Canvas 19
4.5 Market / User Adoption Potential 19
4.6 Cost-Benefit Analysis 20
4.7 Revenue Generation and Pricing Model 20
4.8 Business Feasibility and Viability 21
4.9 Scalability and Long-Term Sustainability 21
4.10 SDG Impact Pathway and Measurable Indicators 21
4.11 Summary 22
CHAPTER 5: SYSTEM DESIGN AND METHODOLOGY 23
5.1 Proposed Solution / System Overview 23
5.2 System Architecture 23
5.3 Module Description 25
5.4 Workflow / Process Design 25
5.5 Data Design / Database Design 26
5.6 UML / DFD / ER Diagrams 28
5.7 Algorithm / Model / Business Logic Design 29
5.7.1 Note Parsing and Extraction 29
5.7.2 Cross-Script Matching 30
5.7.3 Workflow Rule Evaluation 30
5.7.4 Business-Health Score 31
5.8 User Interface and Interaction Design 31
5.9 Security, Privacy, Ethics and Responsible Use 32
5.10 Methodology and Evaluation Strategy 33

CHAPTER 6: IMPLEMENTATION AND INTEGRATION 34
6.1 Development Environment and Technology Stack 34
6.2 Implementation Approach 34
6.3 Module-wise Implementation 35
6.3.1 Smart Input 35
6.3.2 Reading Order Slips 35
6.3.3 Inbound Email and Telegram 35
6.3.4 Event Outbox and Worker 35
6.3.5 Workflow Engine 36
6.3.6 Payments and Business Health 36
6.3.7 Authentication and Access Control 36
6.3.8 Reports and Invoices 36
6.4 Database / API / Model Integration 36
6.5 Key Screens / System Demonstration 37
6.6 Integration and Deployment 44
6.7 Challenges Encountered and Solutions 44
CHAPTER 7: TESTING, RESULTS AND DISCUSSION 45
7.1 Testing Strategy 45
7.2 Test Environment and Test Data 45
7.3 Test Cases and Test Results 45
7.4 Functional and Non-Functional Testing 48
7.5 Performance / Model Evaluation 48
7.5.1 Typed Notes 48
7.5.2 Handwritten Order Slips 49
7.5.3 Consistency and Latency 49
7.5.4 Out-of-Vocabulary Notes 49
7.5.5 Throughput Under Load 50
7.6 Results and Observations 51
7.7 Comparison with Existing / Baseline Approach 52
7.8 Achievement of Project Objectives 52
7.9 Business Model Validation and User Feedback 52
7.10 Business, Social and Environmental Impact 53
7.11 Contribution to Selected SDG Targets 53
7.12 Measurement of SDG Outcomes and Impact Indicators 53
7.13 Limitations and Threats to Validity 54
CHAPTER 8: CONCLUSION AND FUTURE ENHANCEMENTS 55
8.1 Summary of the Project 55
8.2 Conclusions 55
8.3 Achievement of Project Objectives 55
8.4 Key Technical and Business Contributions 56
8.5 Future Scope and Enhancements 56

REFERENCES.....57
PUBLICATIONS: Published Journal/Conference/Patent.....59

LIST OF FIGURES

Fig. 1.1 Informal order channels and the functional areas of a small trading business 1
Fig. 1.2 SDGs addressed by SmartSME and how the platform contributes 5
Fig. 3.1 Stakeholder map 11
Fig. 3.2 Use case diagram of SmartSME 12
Fig. 3.3 Project timeline (Gantt chart) 15
Fig. 3.4 Risk matrix 16
Fig. 4.1 Business Model Canvas for SmartSME 19
Fig. 4.2 SDG impact pathway 21
Fig. 5.1 Overall architecture of SmartSME 23
Fig. 5.2 Layered architecture and modules 24
Fig. 5.3 Smart Input pipeline 25
Fig. 5.4 Inbound order flow for email and Telegram 26
Fig. 5.5 Transactional outbox and event dispatch 26
Fig. 5.6 Entity relationship diagram 27
Fig. 5.7 Data flow diagram, level 0 (context) 28
Fig. 5.8 Data flow diagram, level 1 28
Fig. 5.9 Sequence diagram: recording a sale from a typed note 29
Fig. 5.10 State diagram of an outbox event 29
Fig. 5.11 Five-pass matching of names and products 30
Fig. 5.12 Composition of the business-health score 31
Fig. 5.13 Navigation map of the user interface 32
Fig. 6.1 Technology stack 34
Fig. 6.2 Sign-in page 37
Fig. 6.3 Dashboard with KPI cards, revenue chart and business-health score 37
Fig. 6.4 Smart Input: a note typed in Kannada or Hindi 38
Fig. 6.5 Smart Input: confirmation screen with matched party and products 38
Fig. 6.6 Smart Input: photo of a handwritten order slip and the resulting draft 39
Fig. 6.7 Sales list with source badges and payment status 39
Fig. 6.8 Printable invoice 40
Fig. 6.9 Record payment dialog with payment modes 40
Fig. 6.10 Products page with low-stock indicators 41
Fig. 6.11 Inbox with orders received by email and Telegram 41
Fig. 6.12 Telegram chat sending an order to the SmartSME bot 42
Fig. 6.13 Workflow rules page with execution history 42
Fig. 6.14 Event bus page showing events and their authors 43
Fig. 6.15 Reports page with charts and PDF/CSV download 43
Fig. 6.16 Team page: inviting a member and assigning a role 44
Fig. 6.17 Deployment diagram 44
Fig. 7.1 Automated tests by layer 46

Fig. 7.2 Terminal output of the pytest run showing the per-layer summary 47
Fig. 7.3 Exact-match accuracy on typed notes and handwritten slips 49
Fig. 7.4 Results on out-of-vocabulary notes 50
Fig. 7.5 Throughput and 95th-percentile latency after each optimisation 51
Fig. 7.6 Locust run summary or evaluation harness output 51
Fig. 8.1 Roadmap of future enhancements 56

LIST OF TABLES

Table 1.1 Scope of the project 4
Table 1.2 Selected SDG targets 5
Table 1.3 Alignment of project objectives with SDG targets 5
Table 2.1 Comparison of technical approaches 9
Table 2.2 Comparison with general categories of small-business software 9
Table 3.1 Problem and opportunity analysis 12
Table 3.2 Functional requirements 13
Table 3.3 Non-functional requirements 13
Table 3.4 Hardware and software requirements 14
Table 3.5 Project milestones 15
Table 3.6 Task allocation (to be completed by the team) 16
Table 3.7 Risk register 17
Table 4.1 Customer segments 18
Table 4.2 Customer problems and the value SmartSME offers 19
Table 4.3 Indicative monthly running costs (planning assumptions) 20
Table 4.4 Proposed pricing plans 20
Table 4.5 SDG indicators measurable within SmartSME 22
Table 5.1 Module description 25
Table 5.2 Main database tables 27
Table 5.3 Default workflow rules 30
Table 5.4 Roles and permissions 32
Table 6.1 Development tools 34
Table 6.2 Database migrations 36
Table 6.3 Challenges and solutions 44
Table 7.1 Evaluation data sets 45
Table 7.2 Representative test cases 47
Table 7.3 Verification of non-functional requirements 48
Table 7.4 Extraction results on typed notes (n = 70); F1 in % 48
Table 7.5 Exact match by language variant 48
Table 7.6 Results on real handwritten slips (n = 12) 49
Table 7.7 Out-of-vocabulary notes, fully correct drafts 50
Table 7.8 Load test with 200 concurrent users 50
Table 7.9 Achievement of objectives 52
Table 7.10 User feedback log (to be completed by the team) 53
Table 7.11 Contribution to selected SDG targets 53
Table 7.12 Measurement plan for SDG indicators 54
Table 8.1 Key contributions 56

LIST OF ABBREVIATIONS AND ACRONYMS

AI	Artificial Intelligence
API	Application Programming Interface
ASUSE	Annual Survey of Unincorporated Sector Enterprises
CSV	Comma-Separated Values
DFD	Data Flow Diagram
DPDP	Digital Personal Data Protection (Act, 2023)
ECA	Event-Condition-Action
ERP	Enterprise Resource Planning
GDP	Gross Domestic Product
GST / GSTIN	Goods and Services Tax / GST Identification Number
HSN	Harmonized System of Nomenclature
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
KPI	Key Performance Indicator
LLM	Large Language Model
MSME	Micro, Small and Medium Enterprises
NLP	Natural Language Processing
OCR	Optical Character Recognition
OOV	Out of Vocabulary
ORM	Object-Relational Mapping
PBKDF2	Password-Based Key Derivation Function 2
RBAC	Role-Based Access Control
SaaS	Software as a Service
SDG	Sustainable Development Goal
SKU	Stock Keeping Unit
SME	Small and Medium Enterprise
SPA	Single-Page Application
SQL	Structured Query Language
UI	User Interface
UPI	Unified Payments Interface
UTR	Unique Transaction Reference
UUID	Universally Unique Identifier

LIST OF SYMBOLS / NOTATIONS

A	Sales-weighted product availability (0 to 1)
c	Confidence factor for revenue growth
C_in, C_out	Cash received and cash paid over 90 days
C_fixed	Fixed monthly running cost
c_var	Variable cost per paying shop per month
D	Dead-stock share by value (0 to 1)
g	Revenue growth relative to the baseline month
H	Overall business-health score (0 to 100)
M	Net margin over 90 days
m	Cash movement measure (0 to 1)
n	Number of sales in the revenue baseline
N_be	Number of paying shops needed to break even
p	Average monthly price per paying shop
P, R, F1	Precision, recall and F1 score
S_i	Score of health part i (0 to 100)
TP, FP, FN	True positives, false positives, false negatives
w_i	Weight of health part i

CHAPTER 1

INTRODUCTION

1.1 Background and Context of the Project

Micro, small and medium enterprises (MSMEs) account for a large share of Indian economic activity. Official estimates place the sector's gross value added at 30.1% of all-India GDP in 2022-23, and MSME-related products made up 45.73% of the country's exports in 2023-24 [1]. Most of these firms are very small. The Annual Survey of Unincorporated Sector Enterprises counted 7.34 crore unincorporated non-agricultural establishments in 2023-24, employing more than 12 crore workers [2].

A large part of their trade still runs on informal channels. A regular customer sends a chat message asking for ten bags of rice, a supplier's delivery is noted on paper, and order slips are written by hand and corrected on the spot. None of these reach an accounting system unless somebody types them in later, and in a busy shop that rarely happens on time. Stock counts, customer dues and supplier payables then drift away from what actually took place. Fig. 1.1 shows the four functional areas of a typical shop and the informal channels that feed them.

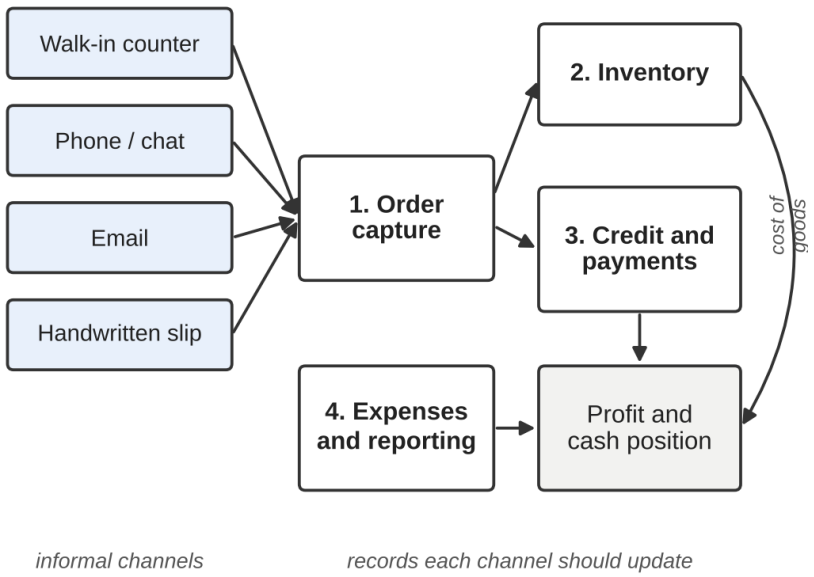


Fig. 1.1 Informal order channels and the functional areas of a small trading business

SmartSME was built to close this gap. It is a web-based business management platform that accepts orders in the forms in which they already arrive, turns them into structured drafts, and records them only after a person in the shop has confirmed them.

1.2 Project Domain Overview

The project draws on three areas of computing.

- Business management and ERP systems for small firms, which keep sales, purchases, inventory, receivables and payables in one database.
- Event-driven architecture, in which every business action publishes an event and other parts of the system react to it, here implemented with a transactional outbox in PostgreSQL.
- AI-assisted data entry, in which language models, OCR and rule-based parsers extract structured fields from free text and images, including code-mixed Hindi, Kannada and English.

The Computer Science and Business Systems programme combines software engineering with business understanding, and SmartSME draws on both: the software has to be reliable and testable, and it also has to fit the way a kirana store or a small wholesaler actually works.

1.3 Motivation

Digital adoption among Indian MSMEs has been uneven. In a 2025 survey of 2,882 MSMEs, 87% made and received payments digitally, but only 12% used enterprise resource planning (ERP) software [3]. One likely reason is that UPI asks very little of the shopkeeper, whereas packaged accounting software still expects every transaction to be typed into a form.

Shop owners already have most of their orders in writing, in chats and on slips, so the information is there to be used. Those notes are often a mix of Kannada, Hindi and English, typed in whichever script the phone keyboard offers, and English-only software handles them poorly. A small business also has nobody to look after a complicated system, so whatever reads these notes has to stay consistent on its own.

1.4 Problem Statement

Small and medium enterprises receive orders through chat messages, email, phone calls and handwritten slips, often in mixed languages and scripts, but existing accounting and ERP software requires each transaction to be entered manually through forms. As a result, records are delayed, incomplete or wrong, and inventory, receivables and payables fall out of step with real activity. The problem addressed by this project is to design and build a system that (i) converts informal text and images into correct structured business records

with human confirmation, and (ii) keeps every recorded transaction consistent with its effects on stock, balances and alerts, without requiring paid services or a separate message broker.

1.5 Aim and Objectives

The aim of the project is to build an AI-assisted, event-driven business management platform that lets small businesses keep accurate books from the informal inputs they already use. The objectives are listed below.

1. **Intelligent input ingestion:** extract structured information from conversational notes and photographed slips, in English, Hindi and Kannada, using AI with an offline fallback.
2. **Asynchronous event bus:** implement a reliable event queue with the transactional outbox pattern on PostgreSQL, so business writes and their consequences never disagree.
3. **Modular ERP core:** model sales, purchases, inventory, payments and analytics as decoupled modules over a normalised, multi-tenant schema.
4. **Configurable automation:** provide a WHEN/THEN rule engine for stock alerts, expense flagging and notifications that can be changed without code.
5. **Dashboard and evaluation:** deliver a responsive web interface with KPIs and a business-health score, and measure extraction accuracy and system performance.

1.6 Scope of the Project

The scope agreed for this phase is summarised in Table 1.1.

Table 1.1 Scope of the project

In scope	Out of scope
Sales, purchases, expenses, payments, products, parties and reports for one or more businesses	Payroll, GST return filing and bank reconciliation
Smart Input from typed notes in English, Hindi and Kannada (native script or Latin letters)	Voice notes and languages other than these three
Reading photographed order slips and screenshots	Scanned multi-page invoices and printed forms in bulk
Orders received by email and Telegram, held in a review queue	WhatsApp Business integration
Event outbox, workflow rules, alerts and an audit trail of who did what	Native mobile apps (the web app is used from phone browsers)
Role-based access for owners, employees and platform admins	Field deployment with real shops (planned as future work)

1.7 Significance and Expected Contribution

- A confirmation-first design, in which nothing from a note, photo, email or chat is written to the books until a person accepts it.
- One parser shared by four entry points (the Smart Input window, uploaded images, email and Telegram), so behaviour is the same wherever an order comes from.
- Matching of names and products across Latin, Devanagari and Kannada script, using a retail lexicon and transliteration instead of machine translation.
- An event backbone in which a record and its event commit in one transaction, giving broker-like guarantees without a broker.
- A measured evaluation of accuracy, consistency, latency and throughput, including a test designed to find where the system fails.

1.8 Industry and Societal Relevance

For industry, SmartSME addresses a segment that packaged ERP has largely missed: shops that are too small for an accountant but large enough to sell on credit to dozens of customers. The platform runs without any paid service, and when an AI key is added it improves extraction without changing how the shop works.

For society, digital records are a step towards formal operation. A shop with clean ledgers can issue proper invoices, track GST, and show a lender how money moves through the business. Supporting regional languages also widens access to people who are not comfortable typing English.

1.9 Sustainable Development Goals (SDGs) Addressed

1.9.1 Identification of Relevant SDGs

The 2030 Agenda for Sustainable Development defines 17 goals and 169 targets [4]. Three goals relate directly to what SmartSME does, as shown in Fig. 1.2.



Fig. 1.2 SDGs addressed by SmartSME and how the platform contributes

1.9.2 Selected SDG Targets

Table 1.2 Selected SDG targets

Target	Text of the target (abridged)	Relevance to SmartSME
8.3	Encourage the formalisation and growth of micro, small and medium-sized enterprises, including through access to financial services	Digital records, invoices and ledgers for shops that keep none today
9.3	Increase the access of small enterprises to financial services, including affordable credit, and their integration into value chains	A dated payment ledger that can show cash flow to lenders and suppliers, with consent
12.3	Halve per capita food waste at the retail and consumer levels	Low-stock alerts and a dead-stock measure that help grocery shops order what sells

1.9.3 Alignment of Project Objectives with SDGs

Table 1.3 Alignment of project objectives with SDG targets

Objective	8.3	9.3	12.3
1. Intelligent input ingestion	Strong	Medium	Low
2. Asynchronous event bus	Medium	Medium	Medium
3. Modular ERP core	Strong	Strong	Medium
4. Configurable automation	Medium	Low	Strong
5. Dashboard and evaluation (health score)	Medium	Strong	Strong

1.9.4 Expected Social, Economic and Environmental Impact

- Social: shopkeepers can work in their own language and script, and staff can record sales without the risk of deleting anything.

- Economic: fewer missed entries, faster collection of dues through overdue tracking, and a record of cash flow that can support credit applications.
- Environmental: better stock visibility reduces over-ordering and expired goods; paper slips remain usable but no longer need to be kept as the only record.

1.10 Constraints and Limitations

- Free tiers of hosted AI services allow roughly fifteen requests per minute, which limits how many notes can be parsed by the model at once.
- The offline parser understands only the verbs, postpositions and products in its vocabulary.
- The evaluation data was written by the team and is synthetic for typed notes; the handwritten slips come from a single writer.
- The health score cannot see the bank balance or the owner's drawings, and uses current purchase prices as the cost of goods.
- The system has not yet been deployed in a working shop.

1.11 Organization of the Report

Chapter 2 reviews existing systems and research. Chapter 3 analyses requirements and presents the project plan and risks. Chapter 4 describes the business model and sustainability framework. Chapter 5 covers the system design and methodology, and Chapter 6 the implementation. Chapter 7 presents testing, results and discussion, and Chapter 8 concludes the report with future enhancements.

CHAPTER 2
LITERATURE REVIEW AND RELATED WORK

2.1 Introduction

This chapter reviews the systems and research that SmartSME builds on. The review covers business software for small firms, event-driven and transactional designs, rule engines, document reading, language models, code-mixed language processing, and ways of measuring the financial health of a business. Each approach is assessed for what it offers a small shop and where it falls short.

2.2 Review of Existing Systems / Research Studies

2.2.1 ERP and Accounting Software for SMEs

ERP systems combine sales, purchasing, inventory and accounting in a shared database, and their coordination benefits are well documented [5]. Reviews of ERP adoption in small and medium enterprises report the same obstacles again and again: licence and implementation cost, customisation effort, and dependence on trained users [6]. Once data has been entered these systems work well. Entry, however, still means filling a form for each transaction, which is the step informal trade skips.

2.2.2 Event-Driven Architectures and Message Brokers

In an event-driven architecture, components publish and consume events instead of calling one another directly [7, 8]. A broker such as RabbitMQ decouples producers from consumers and lets work continue asynchronously. A broker is one more service to deploy and monitor, and by itself it does not solve the dual-write problem: if a sale is saved to the database and its event is sent separately, either step can fail without the other. Distributed transactions across both systems are costly and fragile [9].

2.2.3 Event Sourcing and the Transactional Outbox

Event sourcing stores every change to application state as an event in an append-only log [10]. The transactional outbox pattern is narrower. The business row and a record of its event are written to the same database in one transaction, and a separate relay delivers the event later [11, 12]. Neither can commit without the other, so the dual write disappears. In exchange, the database must also act as a queue, which calls for careful polling, claiming and retry logic.

2.2.4 Rule-Based Automation

Event-condition-action (ECA) rules, studied in active database systems such as HiPAC, fire an action when an event occurs and a condition holds [13]. Production-rule systems based on the Rete algorithm match large rule sets efficiently [14]. Rules of this kind are easy to read and change without code, which suits shops whose policies differ, for example the amount above which an expense should be reviewed. Their limit is that a rule can only test fields the event carries.

2.2.5 Document Reading: OCR and Document-Understanding Models

Character-level OCR engines such as Tesseract are mature and cheap, and read clean printed text well [15]. They return plain characters, so item rows must be rebuilt afterwards, and marks such as strikethroughs are lost. Layout-aware models such as LayoutLM [16] and OCR-free transformers such as Donut [17] understand forms and receipts better, but they need task-specific training data and suitable hardware.

2.2.6 Large Language and Multimodal Models

Large language models can extract structured fields from free text with few or no examples [18], and multimodal models such as Gemini accept images as well as text and can be instructed to return JSON [19]. They are also known to produce fluent output that the input does not support [20], and because they sample their output, the same note can give different answers on different runs. Free tiers are rate limited and depend on a network connection.

2.2.7 Code-Mixed Language Processing and Transliteration

Research on Hindi-English code-mixing has described how speakers switch between languages in online text [21], and benchmarks show that models handle code-switched input less well than monolingual text [22]. Deterministic transliteration libraries convert Devanagari and Kannada to Latin letters by table lookup [23]. Transliteration preserves sound but not meaning: it can tell that अनीता and Anita are one name, but not that चावल means rice.

2.2.8 Measuring Business Health

Classical models of business distress, such as Altman's Z-score, combine ratios taken from audited financial statements [24]. Small shops rarely prepare balance sheets, so a health measure for them has to be built from the records they do keep: sales, purchases, expenses, stock and dated payments.

2.3 Comparative Analysis of Existing Approaches

Table 2.1 compares the technical approaches reviewed above against the needs of a small shop. Table 2.2 compares broad categories of software available to such shops. The product categories are described in general terms and no specific product is evaluated.

Table 2.1 Comparison of technical approaches

Approach	Strength	Weakness for a small shop	Use in SmartSME
Form-based ERP / accounting	Complete, reliable once data is in	Every transaction must be typed	Forms kept as one input path
Message broker (RabbitMQ)	Decoupling, asynchronous work	Extra service; dual write remains	Replaced by outbox; Redis optional
Transactional outbox	Record and event commit together	Database also acts as a queue	Core of the event backbone
ECA / WHEN-THEN rules	Readable, editable without code	Tests only fields an event carries	Workflow engine
Character OCR	Cheap, accurate on clean text	Loses strikethroughs and layout	Fallback image engine
Layout-aware models	Understand receipts and forms	Need training data and GPUs	Not used
Multimodal LLM	Messy notes and handwriting	Hallucination, rate limits, network	Primary engine when a key exists
Transliteration	Offline, deterministic	Sound only, no meaning	Name matching
Z-score style ratios	Established in finance	Need audited statements	Replaced by ledger-based score

Table 2.2 Comparison with general categories of small-business software

Capability	Form-based accounting packages	Digital ledger (khata) apps	SmartSME
Order capture from chat, email, slips	No	Partial (manual)	Yes, with confirmation
Hindi and Kannada notes, any script	Rarely	Interface only	Yes
Reads handwritten order slips	No	No	Yes
Inventory updated by every sale	Yes	Limited	Yes, via events
Configurable automation rules	Limited	No	Yes
Audit trail of who did what	Varies	Limited	Yes
Works without paid AI	Yes	Yes	Yes (offline parser)

2.4 Identified Research / Application Gap

- Existing business software assumes structured entry; none of the reviewed categories turns informal, code-mixed notes and handwritten slips into confirmed records.
- Language models can read such inputs, but their errors and run-to-run variation make them unsafe to write directly into a ledger.
- Small businesses need the consistency guarantees of event-driven systems without operating a broker.
- Health measures for small firms must be built from transaction records, not balance sheets.

SmartSME addresses these gaps by combining human-confirmed AI extraction, cross-script catalogue matching, a PostgreSQL outbox and a ledger-based health score in one system.

2.5 Summary of the Literature Review

The reviewed work provides the building blocks: outbox and ECA patterns for consistency and automation, language models and OCR for reading informal input, and transliteration for names across scripts. None of the reviewed systems combines them for small Indian shops while keeping a person in control of what gets recorded and running without paid infrastructure. The following chapters describe how SmartSME was specified, designed and evaluated to fill that gap.

CHAPTER 3

REQUIREMENTS ANALYSIS AND PROJECT PLANNING

3.1 Introduction

This chapter identifies who uses SmartSME and what they need, states the functional and non-functional requirements, checks feasibility, and presents the project plan, risks and compliance requirements.

3.2 Stakeholder Identification and User Needs

Fig. 3.1 maps the stakeholders and their main needs. The shop owner and counter staff are the primary users; customers interact indirectly by sending orders; platform admins configure businesses without touching their data.

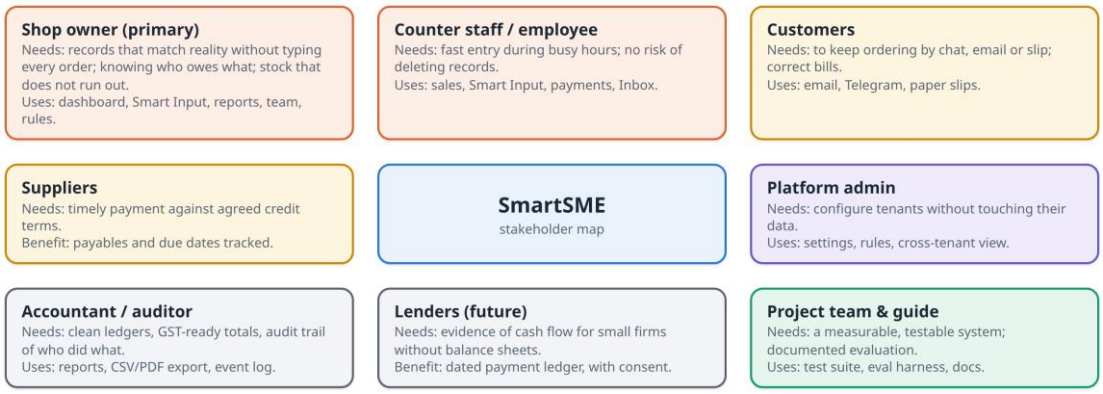


Fig. 3.1 Stakeholder map

The interactions between actors and the system are shown in the use case diagram in Fig. 3.2. Employees can record activity but cannot cancel or delete it; owners have full control of their own business; customers only send orders.

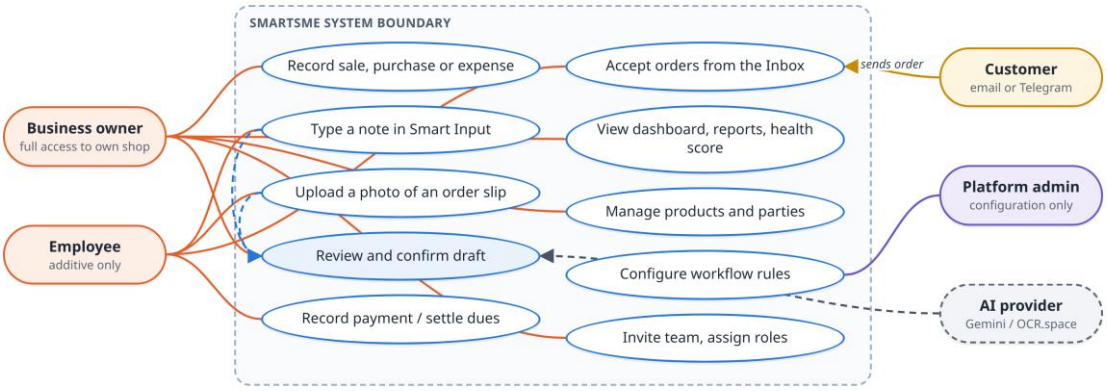


Fig. 3.2 Use case diagram of SmartSME

3.3 Problem / Opportunity Analysis from User and Business Perspectives

Table 3.1 Problem and opportunity analysis

Observation	User perspective	Business opportunity
Orders arrive by chat, email and paper	Typing them again takes time the shop does not have	Capture orders where they already are
Notes mix Kannada, Hindi and English	English-only software is hard to use	Regional-language support as a differentiator
Slips are corrected by hand	A wrong line on a bill causes disputes	Vision model that ignores struck-out items
Sales on credit in both directions	Hard to remember who owes what, since when	Dated payment ledger and overdue tracking
Stock outs and slow-moving goods	Lost sales and cash tied up	Low-stock alerts and dead-stock measure
No accountant, no IT staff	Software must not need maintenance	Single database, no broker, works offline

3.4 Functional Requirements

Table 3.2 Functional requirements

ID	Requirement	Priority
FR1	Users sign up, sign in and join a business by invitation link	High
FR2	Record sales and purchases with line items, discounts, tax, dates and due dates	High
FR3	Record expenses with categories and flag high-value expenses	High
FR4	Maintain products (stock, prices, thresholds) and parties (customers, suppliers)	High
FR5	Parse a typed note in English, Hindi or Kannada into an editable draft	High
FR6	Read a photographed order slip or screenshot into an editable draft	High
FR7	Record nothing from Smart Input until the user confirms the draft	High
FR8	Collect orders from email and Telegram into a review queue	Medium
FR9	Update stock and party balances automatically after every write	High
FR10	Run configurable WHEN/THEN rules and raise notifications	Medium
FR11	Record payments with date and mode (cash, UPI, card, cheque, bank transfer)	High
FR12	Show a dashboard with KPIs, revenue chart and business-health score	Medium
FR13	Generate reports and printable invoices; export PDF and CSV	Medium
FR14	Enforce roles (superuser, admin, owner, employee) on every route	High
FR15	Keep an event log showing who caused each change	Medium

3.5 Non-Functional Requirements

Table 3.3 Non-functional requirements

Attribute	Requirement	How it is verified
Consistency	A record and its event commit together; effects apply exactly once	Dispatch and smoke tests
Reliability	AI failure degrades to the offline parser, never to an error	Mocked AI tests
Security	PBKDF2 passwords, signed httpOnly cookie, RBAC, sign-in throttling	RBAC and throttle tests
Privacy	Only the note or image is sent to an AI provider; full card numbers refused	Payment-mode tests
Performance	Lists paginated; dashboard aggregated in SQL; p95 under 0.5 s at 200 users	Locust load test
Usability	Every draft editable; works in phone browsers; light and dark themes	Manual review
Maintainability	Layered code, migrations, lint and format checks, documentation	Ruff, Alembic, docs
Cost	Runs with no paid service	Offline test run

3.6 Hardware and Software Requirements

Table 3.4 Hardware and software requirements

Item	Requirement
Development machine	64-bit laptop or desktop, 8 GB RAM or more, Windows, macOS or Linux
Runtime (backend)	Python 3.10 or later; FastAPI, SQLAlchemy, Alembic, Celery (optional)
Runtime (frontend)	Node.js with npm; React 19, Vite 6, Tailwind CSS 4
Services	Docker with Compose: PostgreSQL 16, Redis 7 (optional), Mailpit (development mail)
AI services (optional)	Google Gemini API key (free tier); OCR.space key (free tier)
Messaging (optional)	Telegram bot token; POP3 mailbox for inbound orders
Client	Any modern browser on a desktop or phone
Deployment	Static host for the SPA (Vercel), an application host for the API (Render), managed PostgreSQL (Neon)

3.7 Feasibility Analysis

3.7.1 Technical Feasibility

All components are mature open-source tools with large communities. The riskiest part, extraction from informal input, was reduced by keeping an offline parser that works without any external service and by requiring human confirmation of every draft. PostgreSQL supports the atomic claim and row-locking operations that the outbox and the inbound queue rely on.

3.7.2 Economic / Cost Feasibility

Development used only free software. Gemini and OCR.space were used on their free tiers, and Neon, Vercel and Mailpit offer free plans suitable for development and small deployments. The only unavoidable running cost for a production deployment is an application host, discussed further in Chapter 4.

3.7.3 Operational and Schedule Feasibility

A shopkeeper does not have to change how orders are taken; the system adapts to existing habits, and the confirmation screen keeps the person in control. The project was completed within the semester using the plan in Section 3.8. A single Windows script starts the whole stack, which made it practical for team members to run and test the system on their own laptops.

3.8 Project Plan, Milestones and Task Allocation

Development proceeded in four phases, shown in Fig. 3.3. The first prototype was built in TypeScript; the system was then rewritten with a FastAPI backend and a React frontend to support the outbox design, before AI channels, evaluation and payment features were added.

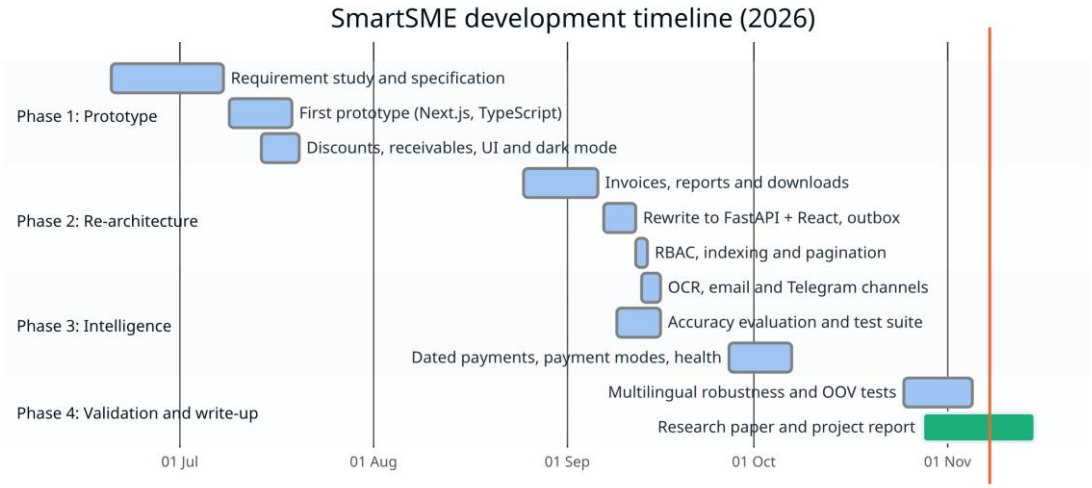


Fig. 3.3 Project timeline (Gantt chart)

Table 3.5 Project milestones

M	Milestone	Completed	Status
M1	Requirement study and specification	July 2026	Done
M2	First working prototype with sales, discounts and receivables	July 2026	Done
M3	FastAPI + React rewrite with transactional outbox	September 2026	Done
M4	RBAC, indexing, pagination and SMTP/POP3 integration	September 2026	Done
M5	OCR, email and Telegram channels; accuracy evaluation	September 2026	Done
M6	Dated payments, payment modes and health score	October 2026	Done
M7	Multilingual robustness tests, research paper and report	November 2026	Done

Table 3.6 Task allocation (to be completed by the team)

Work package	Team member(s)
Requirements, specification and objectives	[Name]
Frontend pages, UI theme and invoices	[Name]
Backend domain layer, payments and reports	[Name]
Smart Input, OCR and multilingual matching	[Name]
Event outbox, workflow engine and Celery dispatch	[Name]
Inbound email and Telegram channels	[Name]
Testing, evaluation and load testing	[Name]
Documentation, paper and report	[Name]

3.9 Risk Assessment and Mitigation

Risks were rated by likelihood and impact (Fig. 3.4) and each was given a mitigation (Table 3.7).

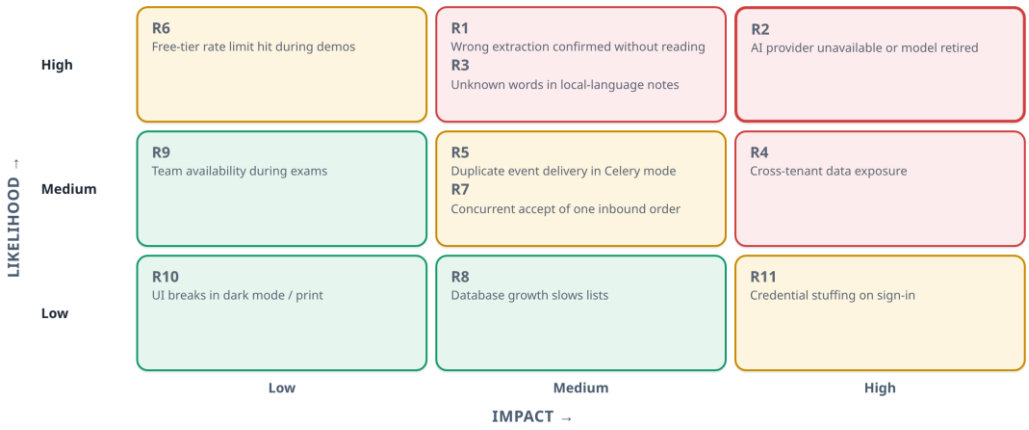


Fig. 3.4 Risk matrix

Table 3.7 Risk register

ID	Risk	Mitigation
R1	Wrong extraction confirmed without reading	Confirmation screen shows every field; engine name shown; source badge on each record
R2	AI provider unavailable or model retired	Offline parser fallback; circuit breaker skips a failing provider for two minutes; model name configurable
R3	Unknown words in local-language notes	Lexicon and transliteration; English product name from the model; unmatched lines left blank, not guessed
R4	Cross-tenant data exposure	business_id on every row; permission check on every route; 404 for other tenants' resources
R5	Duplicate event delivery in Celery mode	Atomic claim; loser of the race does nothing
R6	Free-tier rate limit	Pacing in the harness; fallback counted and reported
R7	Two users accept the same inbound order	Row locked FOR UPDATE; second accept gets 409 Conflict
R8	Data growth slows lists	Tenant indexes; pagination; SQL aggregates
R9	Team availability during exams	Phased plan; documentation so any member can run the stack
R10	UI breaks in dark mode or print	Print stylesheet forces the light theme; frontend tests
R11	Credential stuffing	Per-account and per-IP throttling stored in PostgreSQL

3.10 Data Privacy, Security, and Compliance Requirements

- Personal data of customers and suppliers (names, phone numbers) is held per business and never shared across tenants. Processing should follow the Digital Personal Data Protection Act, 2023 [25].
- Passwords are stored with PBKDF2-HMAC-SHA256 at 100,000 iterations [26]; sessions use a signed JSON Web Token in an httpOnly cookie [27].
- For card payments only the last four digits are accepted; longer numbers are refused, not trimmed.
- When an AI key is configured, only the note or image being parsed is sent to the provider. No catalogue, customer list or history leaves the server. Without a key, nothing leaves the machine.
- Invitation tokens are stored only as SHA-256 hashes, last seven days and can be used once.
- Records needed for GST (GSTIN, HSN, tax totals) are stored with each business and document so they can be produced on request.

CHAPTER 4

BUSINESS MODEL AND SUSTAINABILITY FRAMEWORK

4.1 Introduction to the Proposed Business / Solution

SmartSME can be offered as a hosted software service (SaaS) for small trading businesses. A shop signs up, adds its products and regular parties, and keeps taking orders the way it already does; the platform turns those orders into confirmed records and keeps stock, dues and alerts up to date. This chapter sets out who the customers are, what value they receive, how the service could earn revenue, and how it supports the SDG targets identified in Chapter 1. The financial figures in this chapter are planning assumptions made by the team, not measured market data.

4.2 Identification of Target Users and Customer Segments

Table 4.1 Customer segments

Segment	Characteristics	Main need
Kirana and general stores	High order volume, many regular customers on credit, orders by phone and chat	Fast entry, dues tracking, stock alerts
Small wholesalers and distributors	Bulk orders, handwritten slips from retailers, supplier credit	Reading slips, purchase tracking, payables
Hardware and building-material shops	Large tickets, items by unit and weight, credit sales	Invoices, receivables, product catalogue
Micro manufacturers selling on credit	Few products, many buyers, purchase of raw material	Cost tracking, cash-flow visibility

The initial focus is Karnataka, where notes often combine Kannada, Hindi and English and the team can support onboarding in person.

4.3 Customer Problem and Value Proposition

Table 4.2 Customer problems and the value SmartSME offers

Customer problem	Value proposition
Orders sit in chats and on slips and are typed in late or never	Paste or photograph the order; a draft is ready to confirm in seconds
English-only software is hard for staff to use	Notes in Kannada, Hindi or English, in native script or Latin letters
Mistakes on bills cause disputes	Every draft is reviewed before it is recorded; struck-out items are ignored by the vision engine
Stock and dues do not match reality	Every confirmed record updates stock and balances through the event chain
No time to analyse the business	A health score out of 100 with plain labels: Healthy, Fair, At risk
Software subscriptions are costly	Core features work with no paid service

4.4 Business Model Canvas

The business model is summarised using the Business Model Canvas [28] in Fig. 4.1.

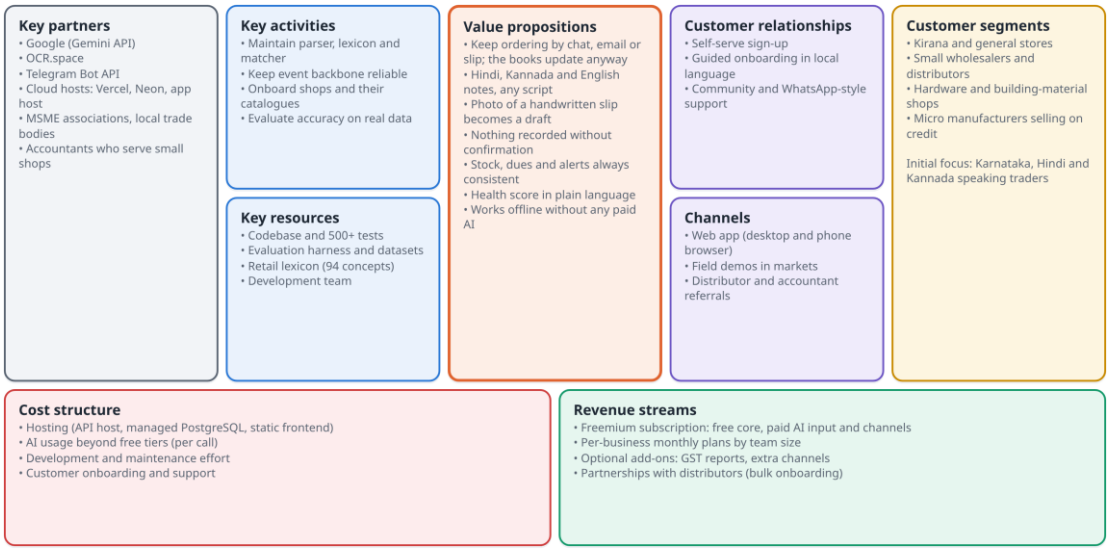


Fig. 4.1 Business Model Canvas for SmartSME

4.5 Market / User Adoption Potential

India had an estimated 7.34 crore unincorporated non-agricultural establishments in 2023-24 [2], and MSMEs contribute about 30% of GDP [1]. Readiness for digital tools is also high: 87% of surveyed MSMEs already use digital payments, while only 12% use ERP software [3]. The gap between those two figures is the space SmartSME targets.

Adoption barriers reported for MSMEs include lack of awareness, skill shortages, high upfront cost and concerns about reliability [3]. The adoption plan responds to each:

demonstrations in local markets and through distributors, an interface that needs no training beyond confirming a draft, a free tier, and an offline mode that keeps working when an AI service is down.

4.6 Cost-Benefit Analysis

Table 4.3 lists the expected monthly running costs of a small hosted deployment serving a few hundred shops. The amounts are indicative assumptions for planning and should be replaced with current quotes before any launch.

Table 4.3 Indicative monthly running costs (planning assumptions)

Cost item	Assumed monthly cost	Notes
Application host (API and worker)	₹1,500	Small instance; scale out with more workers
Managed PostgreSQL	₹0 to ₹1,500	Free plan at first, paid plan as data grows
Static frontend hosting	₹0	Free plan for static sites
AI usage beyond free tier	₹1,000	Only for shops on paid plans
Domain, email, monitoring	₹500	
Total (C_fixed)	About ₹4,500	Excludes team salaries

Benefits for a shop are mainly time and accuracy. If a shop records 40 orders a day and confirming a draft saves even one minute per order compared with typing it into a form, the shop saves around 40 minutes daily. Fewer missed entries, earlier collection of overdue payments and fewer stock-outs add to this, although these effects still have to be measured in a field trial.

4.7 Revenue Generation and Pricing Model

A freemium subscription is proposed, so that a shop can start without paying and upgrade when it relies on AI input or several staff accounts.

Table 4.4 Proposed pricing plans

Plan	Price (proposed)	Includes
Free	₹0	One user; forms, offline Smart Input parser, inventory, reports, health score
Pro	₹299 per month	Up to 3 users; AI note parsing and slip reading; email and Telegram inbox
Business	₹799 per month	Up to 10 users; roles and audit log; priority support; data export

Additional revenue could come from add-ons such as GST-ready report packs, and from partnerships with distributors who onboard their retailers in bulk.

4.8 Business Feasibility and Viability

The number of paying shops needed to cover running costs is given by Eq. (4.1), where C_fixed is the fixed monthly cost, p the average monthly price per paying shop, and c_var the variable cost per paying shop (mainly AI usage).

$$N_be = C_fixed / (p - c_var) \quad (4.1)$$

Taking C_fixed = ₹4,500 from Table 4.3, an average price p = ₹399 (a mix of Pro and Business plans) and c_var = ₹20 per shop, N_be = 4,500 / 379 ≈ 12 paying shops. Even with costs several times higher, break-even needs only tens of paying customers, so the model is viable at small scale. The larger cost is the team's time for development, onboarding and support, which would need funding or revenue beyond break-even.

4.9 Scalability and Long-Term Sustainability

Technically, the system scales by adding API worker processes and, if needed, Celery workers fed through Redis, while PostgreSQL remains the single source of truth. Tenant indexes and pagination keep queries fast as data grows (Chapter 7). Each business is isolated by a column, so new shops add rows, not databases.

Commercially, sustainability depends on keeping the free tier cheap to serve, which the offline parser makes possible, and on converting shops that come to depend on AI input and the inbox. Extending the lexicon through user corrections would let the product improve for each new region without additional engineering.

4.10 SDG Impact Pathway and Measurable Indicators

Fig. 4.2 traces how the platform's inputs and activities lead to outcomes and to the SDG targets. Table 4.5 lists the indicators and how each can be measured from data the system already stores.

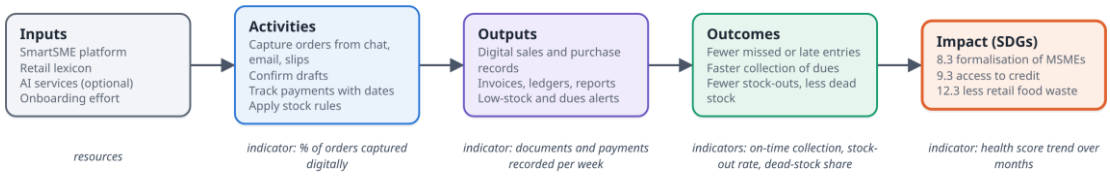


Fig. 4.2 SDG impact pathway

Table 4.5 SDG indicators measurable within SmartSME

Target	Indicator	Source in SmartSME
8.3	Share of a shop's orders recorded digitally; number of invoices issued per month	Sales and purchases tables, source field
9.3	Share of credit collected by its due date; months of dated payment history	Payments ledger and due dates
12.3	Stock-out rate of fast-moving items; dead-stock share by value	Inventory part of the health score
All	Overall health score trend	Dashboard, monthly

4.11 Summary

SmartSME targets small trading businesses that already take orders in writing but do not keep structured records. Its value lies in capturing those orders with little effort, in the shop's own language, while keeping a person in control. A freemium subscription can cover running costs with a small number of paying shops, and the indicators needed to show SDG impact are already produced by the system's own data.

CHAPTER 5

SYSTEM DESIGN AND METHODOLOGY

5.1 Proposed Solution / System Overview

SmartSME is a modular monolith: one FastAPI application, one PostgreSQL database and one React single-page application, with an event bus inside the database. The design follows four criteria drawn from the literature review and the requirements.

1. Customers keep ordering the way they already do, so the system accepts plain text, photographs, email and chat.
2. Nothing from outside is written to the books without a person's confirmation.
3. The system runs with no paid service and degrades gracefully when an AI provider is absent.
4. A single database is both the system of record and the event queue, removing the dual write and the need for a broker.

5.2 System Architecture

Fig. 5.1 shows the overall architecture. The browser talks to the API over a cookie-authenticated JSON interface. Requests go from routers to a domain layer that holds the business rules. Every write stores its rows and its event in one transaction; the event worker then applies the consequences. Inbound email and Telegram messages enter through a collector and wait in a review queue.

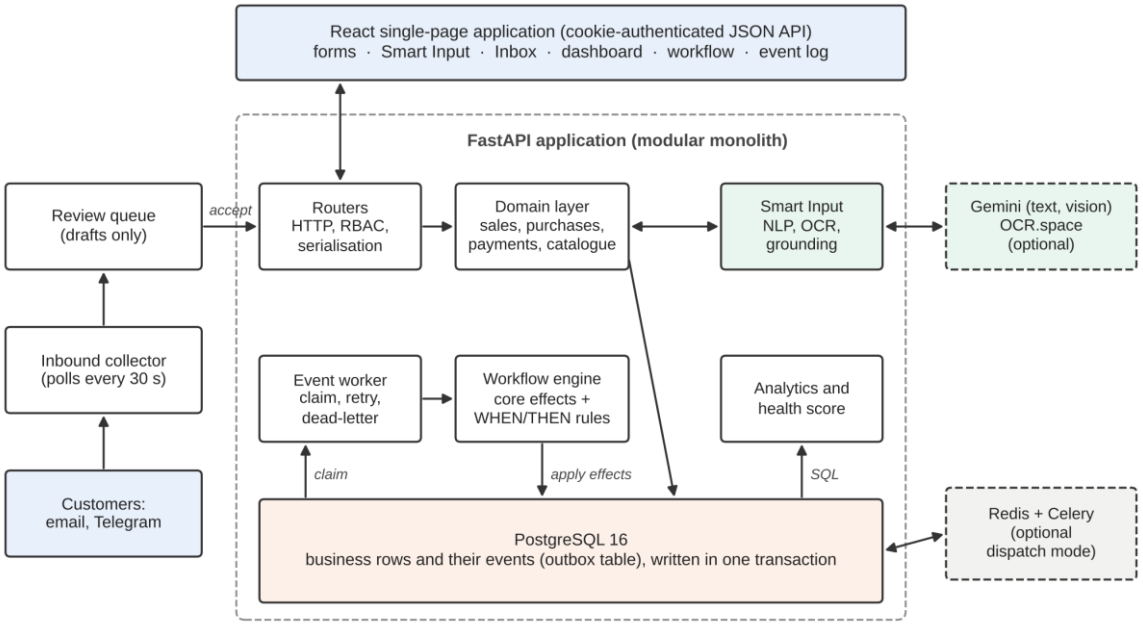


Fig. 5.1 Overall architecture of SmartSME

The layered structure of the codebase is shown in Fig. 5.2. Routers handle HTTP parsing, permissions and serialisation; the domain layer knows nothing about HTTP; services such as Smart Input and the workflow engine sit beside the domain layer; and all persistent state lives in PostgreSQL behind SQLAlchemy.

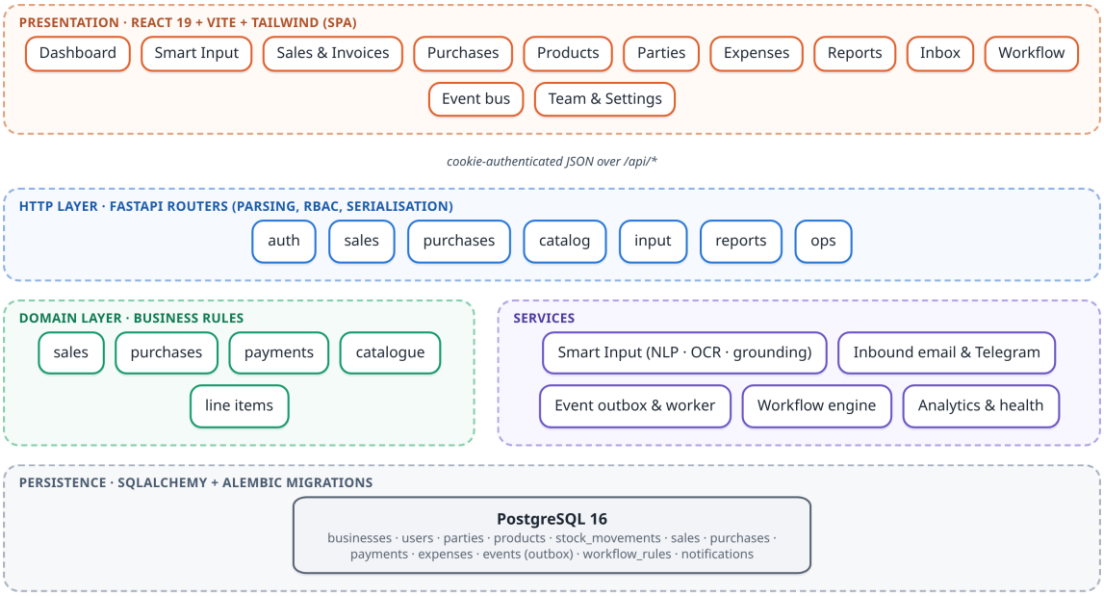


Fig. 5.2 Layered architecture and modules

5.3 Module Description

Table 5.1 Module description

Module	Responsibility	Main files
Authentication and roles	Sign-up, sign-in, sessions, invitations, RBAC, throttling	core/security.py, core/roles.py, invites.py
Catalogue	Products, parties, stock movements	domain/catalog.py, models/
Sales and purchases	Documents, line items, discounts, tax, due dates, cancellation	domain/sales.py, domain/purchases.py
Payments	Dated payment ledger, payment modes, settlements	domain/payments.py
Smart Input	Note parsing, image reading, grounding against the catalogue	ai/, smart_input.py
Inbound channels	Email (POP3) and Telegram collection, review queue	inbound/
Event bus	Outbox publish, claim, retry, dead-letter, Celery dispatch	events.py, worker.py, tasks.py
Workflow engine	Core effects and WHEN/THEN rules, executions log	workflow.py
Analytics and health	Dashboard aggregates, revenue series, health score	analytics.py, health.py
Reports	Report builder, PDF and CSV export	reports.py
Frontend	Pages, components, API client, themes	frontend/src/

5.4 Workflow / Process Design

All informal input goes through the Smart Input pipeline (Fig. 5.3). A typed note, an email or Telegram message, or a photograph passes through extraction and grounding to a confirmation screen, and only a confirmed draft reaches the domain layer.

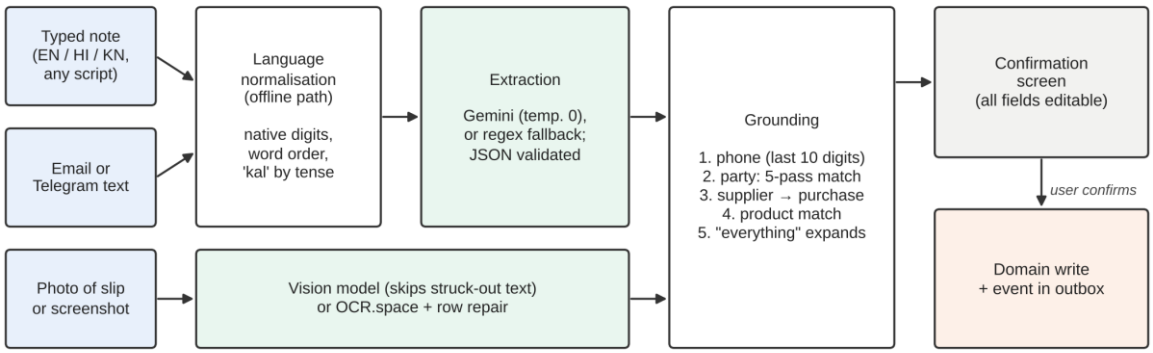


Fig. 5.3 Smart Input pipeline

Orders that arrive from outside follow the flow in Fig. 5.4. They are parsed by the same engine but held as drafts in a review queue. Accepting a draft locks its row, so if two staff members click at once the order is still recorded only once.

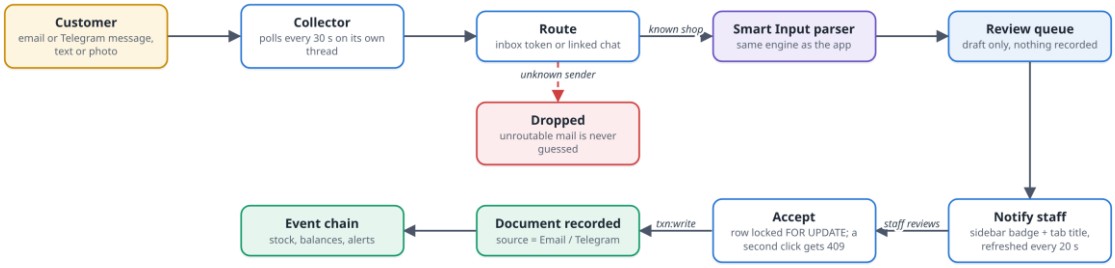


Fig. 5.4 Inbound order flow for email and Telegram

Once a draft is confirmed, the transactional outbox (Fig. 5.5) carries its consequences. The business rows and the event commit together; in inline mode the chain is applied before the response returns, and in Celery mode workers apply it shortly afterwards.

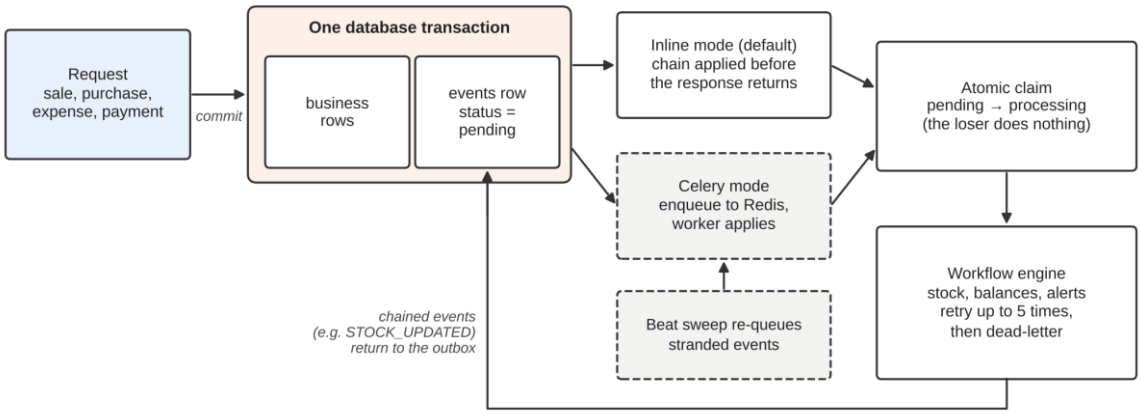


Fig. 5.5 Transactional outbox and event dispatch

5.5 Data Design / Database Design

The schema is normalised and every tenant-owned table carries a business_id. Fig. 5.6 shows the main entities and relationships, and Table 5.2 summarises the purpose of each table.

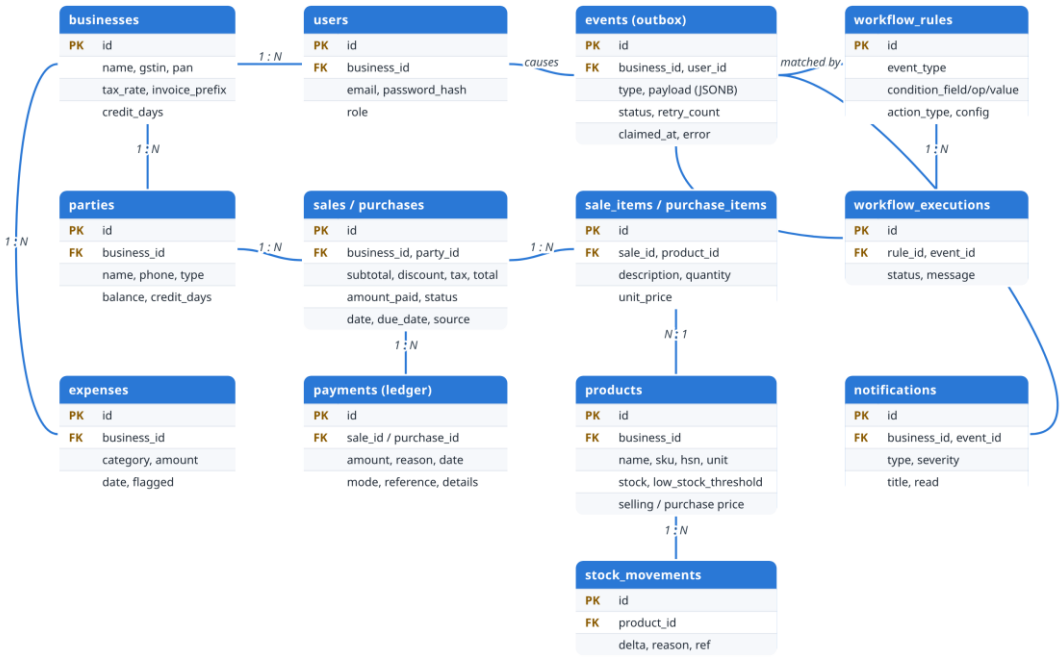


Fig. 5.6 Entity relationship diagram

Table 5.2 Main database tables

Table	Purpose
businesses	Tenant root: name, GSTIN, PAN, address, tax rate, invoice prefix, default credit days
users	Email, PBKDF2 password hash, role, business
parties	Customers and suppliers with running balance and optional credit days
products	Stock, unit, HSN/SKU, purchase and selling price, low-stock threshold
stock_movements	Append-only inventory ledger (delta, reason, reference)
sales, sale_items	Invoices with subtotal, discount, tax, total, amount paid, date, due date, source
purchases, purchase_items	Supplier bills with the same structure as sales
payments	Append-only payment ledger: amount, reason, date, mode, reference
expenses	Category, description, amount, date, flagged marker
events	The outbox: type, JSONB payload, status, retry count, claim time, author
workflow_rules, workflow_executions	Rule definitions and the audit trail of their evaluation
notifications	Alerts shown in the bell menu, linked to their rule and event

5.6 UML / DFD / ER Diagrams

The context-level data flow diagram (Fig. 5.7) shows SmartSME exchanging data with shop users, customers, messaging services and AI providers. The level-1 diagram (Fig. 5.8) breaks the system into five processes and four data stores.

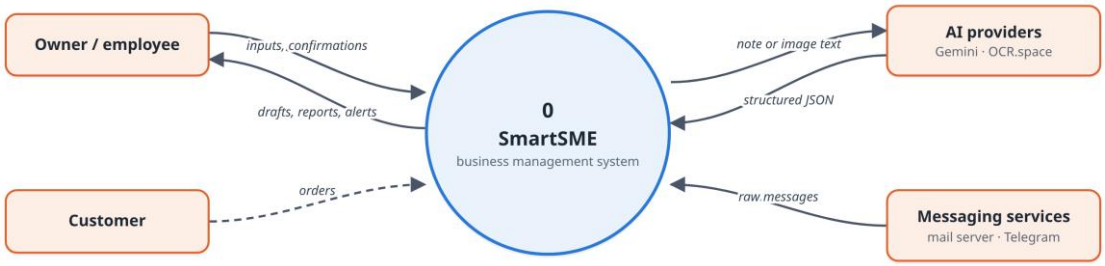


Fig. 5.7 Data flow diagram, level 0 (context)

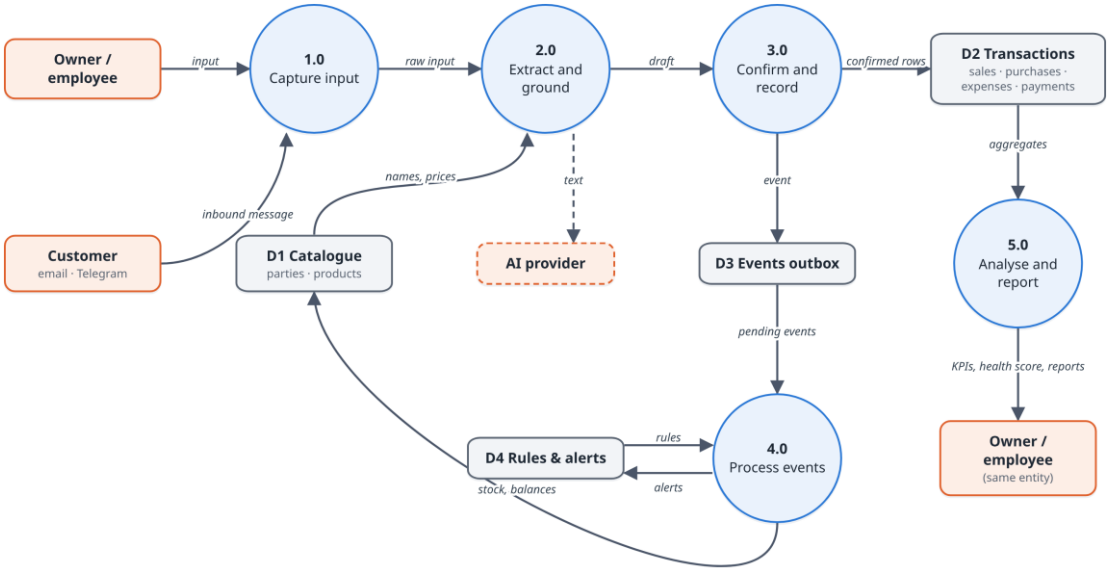


Fig. 5.8 Data flow diagram, level 1

The sequence of calls when a shopkeeper records a sale through Smart Input is shown in Fig. 5.9. The life of an event in the outbox is shown as a state diagram in Fig. 5.10.

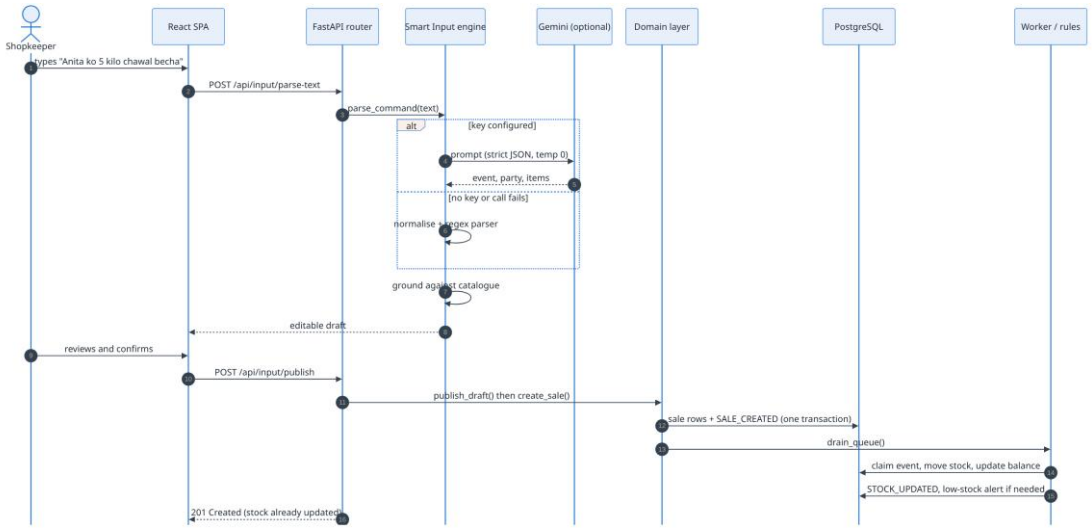


Fig. 5.9 Sequence diagram: recording a sale from a typed note

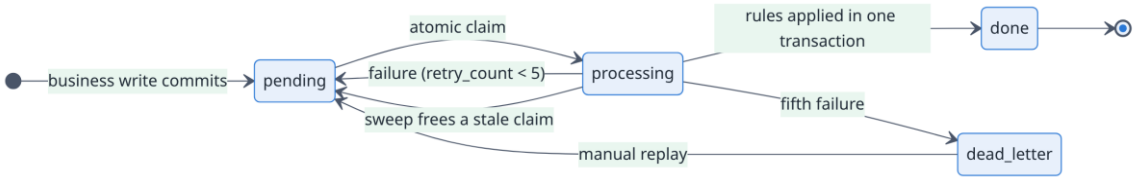


Fig. 5.10 State diagram of an outbox event

5.7 Algorithm / Model / Business Logic Design

5.7.1 Note Parsing and Extraction

The parsing algorithm is summarised below.

1. If a Gemini key is configured and the provider is healthy, send one prompt asking for strict JSON (event type, party, line items with an English name for each product, amount, category, discount, date), with temperature fixed at zero.
2. Extract the first JSON object from the reply and validate every field. If the call fails or the reply is unusable, fall back to step 3. After three consecutive failures the provider is skipped for two minutes.
3. **Offline path:** rewrite Hindi and Kannada into English word order (native digits to numbers, postpositions moved before names, "kal" resolved by tense), then run the regular-expression parser. Line items are split on quantities, not on the word "and".
4. **Ground the result:** match the phone number (last ten digits) for slips, match the party and each product with the five-pass matcher, and let a matched supplier turn the draft into a purchase.
5. Return the draft for confirmation. Nothing is written.

5.7.2 Cross-Script Matching

Names and products are compared on a folded key that ignores what writers disagree about (diacritics, doubled letters, aspirates, long vowels, w/v). The matcher then tries five passes from certain to merely likely (Fig. 5.11). A retail lexicon of 94 concepts links words such as चावल, chawal and ಅಕ್ಕಿ to "rice"; transliteration links names across scripts [23].

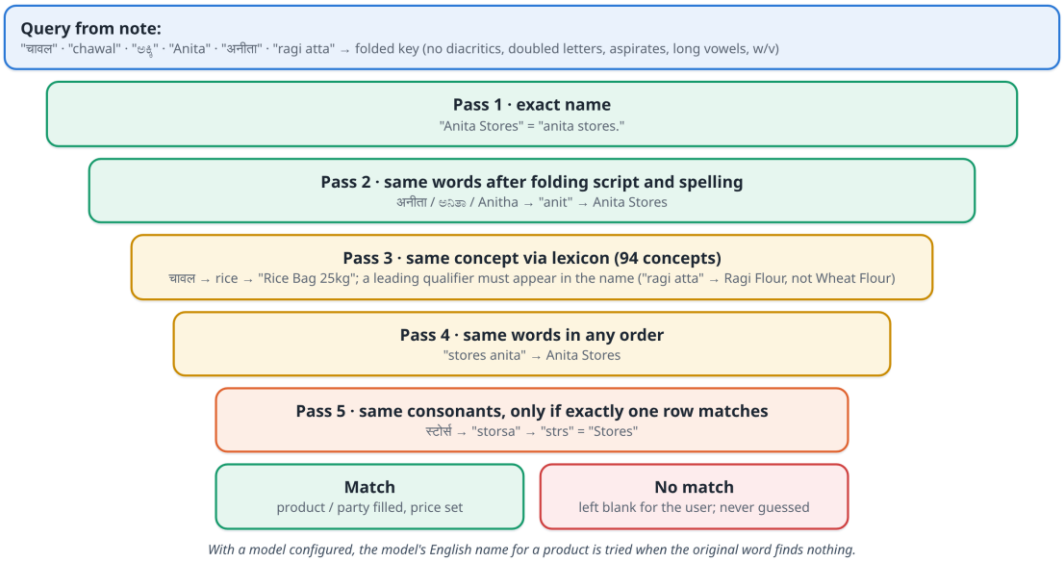


Fig. 5.11 Five-pass matching of names and products

5.7.3 Workflow Rule Evaluation

Stock and balance updates are core effects that always run. Configurable behaviour is expressed as rules with an event type, an optional condition (field, operator, value) and an action. Supported operators are greater than, greater or equal, less than, less or equal, equal and not equal. A condition on a field the event does not carry evaluates to false. Table 5.3 lists the rules each new business starts with.

Table 5.3 Default workflow rules

Rule	When	Then
Update inventory on sale	SALE_CREATED	update_inventory
Low-stock restock alert	STOCK_UPDATED	restock_alert
Flag high-value expense	EXPENSE_ADDED and amount > 10000	flag_expense
Unpaid sale reminder	SALE_CREATED and payment status = unpaid	notify

5.7.4 Business-Health Score

The health score combines four parts, each scored from 0 to 100 (Fig. 5.12). Cash flow averages on-time collection, on-time supplier payment and cash movement, where cash movement is given by Eq. (5.1), clipped to the range 0 to 1.

$$m = 0.5 + 2.5 \times (C_in - C_out) / C_in \tag{5.1}$$

Profitability uses the net margin M over 90 days, net of GST:

$$S_profit = 40 + 600 \times M \tag{5.2}$$

Revenue compares the last 30 days with the average month of the previous 90 days. The growth g is scaled by a confidence factor c that grows with the number of baseline sales n:

$$S_rev = 75 + 100 \times g \times c, \quad c = min(1, n / 20) \tag{5.3}$$

Inventory combines sales-weighted availability A and the dead-stock share D:

$$S_inv = 100 \times (0.7\, A + 0.3\, (1 - D)) \tag{5.4}$$

Parts without data are dropped and the weights w_i (30, 30, 25, 15) are rescaled; no score is shown unless at least half the weight has data. The overall score H is capped at 30 points above the weakest part:

$$H = min(\Sigma\, w_i\, S_i / \Sigma\, w_i\, ,\ \ min_i\, S_i + 30\,) \tag{5.5}$$

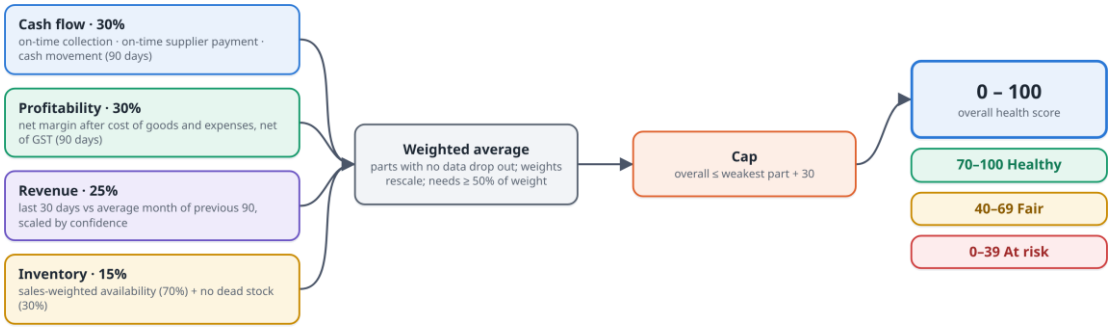


Fig. 5.12 Composition of the business-health score

5.8 User Interface and Interaction Design

The interface is organised around the dashboard, from which the user reaches capture, books and insight pages (Fig. 5.13). Pages that change configuration or the team are hidden from employees, and every rule is enforced again on the server.

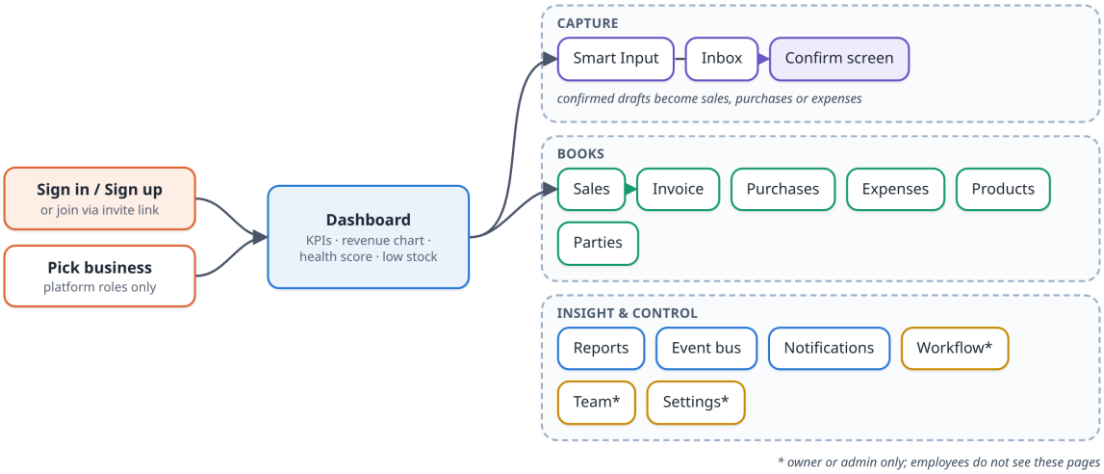


Fig. 5.13 Navigation map of the user interface

- The confirmation screen shows every extracted field as an editable input, along with the engine that produced it.
- Each sale and purchase carries a source badge (Form, AI · Text, AI · Photo, Email, Telegram), so the origin of every record is visible.
- Payment status reads Overdue once an unpaid balance passes its due date.
- Light and dark themes are supported; printing an invoice switches to the light theme automatically.
- A pending-order count appears in the sidebar and the browser tab title on every page.

5.9 Security, Privacy, Ethics and Responsible Use

Access is controlled by four roles and eight permissions [29]. Table 5.4 shows which role holds which permission. An unknown role receives no permissions, and a request for another tenant's resource returns 404 so that tenants cannot discover one another.

Table 5.4 Roles and permissions

Permission	Superuser	Admin	Owner	Employee
data:read	✓	✓	✓	✓
txn:write (record activity)	✓		✓	✓
data:manage (cancel, delete, back-date)	✓		✓	
catalog:write	✓		✓	
config:write (profile, tax, rules)	✓	✓	✓	
users:manage	✓		✓	
events:operate (replay, drain)	✓		✓	
platform:cross-tenant	✓	✓		

- Sign-in throttling: five failures lock an account (employees excepted) for fifteen minutes, doubling with further failures up to a day; twenty failures from one IP address across all accounts in fifteen minutes are also blocked.
- Responsible use of AI: extraction is a suggestion. The user confirms every draft, the engine is named on screen, and evaluation results, including failures, are published with the project.
- Data minimisation: AI providers receive only the note or image being parsed.

5.10 Methodology and Evaluation Strategy

The project followed an iterative, incremental method. Each feature was built, tested with automated tests, reviewed and documented before the next one. Accuracy was measured separately from the test suite with an evaluation harness that runs the same functions the application uses against inputs with gold labels written in advance. Fields are scored as an information-extraction task [30]: a wrong answer counts as both a false positive and a false negative.

$$P = TP / (TP + FP) \tag{5.6}$$

$$R = TP / (TP + FN) \tag{5.7}$$

$$F1 = 2 P R / (P + R) \tag{5.8}$$

Exact match is the share of inputs in which every scored field is correct, which is the share of drafts a shopkeeper could accept without editing. Performance was measured with Locust against a seeded business, and database plans were checked with EXPLAIN ANALYZE.

CHAPTER 6

IMPLEMENTATION AND INTEGRATION

6.1 Development Environment and Technology Stack

Fig. 6.1 summarises the technology stack. Development was done on Windows laptops with Docker running PostgreSQL, Redis and Mailpit; a PowerShell script (run-dev.ps1) creates the virtual environment, applies migrations and starts the API and the web app.



Fig. 6.1 Technology stack

Table 6.1 Development tools

Tool	Use
Visual Studio Code	Code editing for backend and frontend
Git and GitHub	Version control and collaboration
Docker Compose	PostgreSQL 16, Redis 7 and Mailpit for local development
pytest, Vitest	Backend and frontend automated tests
Ruff, Prettier	Linting and formatting
Locust	Load testing
FastAPI interactive docs	Manual API exploration at /docs

6.2 Implementation Approach

The first prototype, built in July 2026 with Next.js and TypeScript, covered sales, purchases, discounts and receivables. In September the system was rewritten as a Python FastAPI backend and a React single-page application. The rewrite made it possible to put the event bus inside PostgreSQL as a transactional outbox, to share one domain layer between forms and Smart Input, and to write a large automated test suite. The workflow engine was ported from the original TypeScript module.

Each later feature followed the same cycle: design note in the docs folder, migration if the schema changed, domain code, router, frontend page, tests, and finally an evaluation where accuracy mattered.

6.3 Module-wise Implementation

6.3.1 Smart Input

One provider interface wraps Google Gemini, chosen because its free tier needs no card and the same key reads text and images. The prompt asks for strict JSON, and every field is validated before use. Without a key, or after a failure, a regular-expression parser covers the same fields. A normalisation module converts native digits, moves Hindi and Kannada postpositions in front of names, and resolves the Hindi word "kal" by tense. Notes with several items produce several line items, because the parser splits on quantities.

6.3.2 Reading Order Slips

Images are read first by the Gemini vision model, which is told to ignore struck-out text, and otherwise by OCR.space. Both return the same structure (party, phone, document type, line items, total, date, discount). A post-processing step rebuilds item rows from OCR text and corrects a handwritten 1 read as a capital I and two rows merged by a tab. When only OCR.space is available, the draft is labelled with its name so the reviewer checks for cancelled lines.

6.3.3 Inbound Email and Telegram

A background collector polls each configured channel every 30 seconds on its own thread. Each business has a private token that routes mail (orders+token@...) and linked Telegram chats to it. Messages become drafts in a review queue; accepting one requires the txn:write permission and records the channel as the document's source.

6.3.4 Event Outbox and Worker

Every write calls publish() inside the same database session as the business rows, as in the listing below. The worker claims events with a single atomic update, processes them in batches of 20, retries failures and dead-letters an event after five attempts.

```
sale = Sale(...); db.add(sale); db.flush()
db.add_all(items)
publish(db, business_id, "SALE_CREATED", {"saleId": str(sale.id)})
db.commit()      # rows and event together
drain_queue()    # apply the chain before the response returns
```

In Celery mode the same events are handed to Redis. A periodic sweep re-queues events whose enqueue was lost or whose worker died, judging staleness by the time of the claim.

6.3.5 Workflow Engine

Each event is handled in one transaction: core effects (stock movements, party balances) first, then the rules for that event type. Chained events such as STOCK_UPDATED inherit the author of the event that started the chain. Every evaluation is written to workflow_executions and shown on the Workflow page.

6.3.6 Payments and Business Health

Every movement of money is written to the payments ledger with its date and mode. Due dates come from the document, the party or the business default (30 days), in that order. The health module gathers its inputs with SQL and scores them with pure functions, which are tested without a database.

6.3.7 Authentication and Access Control

Passwords are hashed with PBKDF2-HMAC-SHA256 at 100,000 iterations and sessions are signed JWTs in httpOnly cookies valid for 30 days. Routes declare the permission they need, and an owner invites staff through single-use links.

6.3.8 Reports and Invoices

Reports are built with SQL aggregation and exported as PDF (ReportLab) or CSV. Invoices carry the due date and the payments made against them, on screen and on paper.

6.4 Database / API / Model Integration

The schema evolved through nine Alembic migrations (Table 6.2). The API exposes more than 70 routes, and every list endpoint returns a page of rows with an envelope giving the page, page size, total and whether more pages exist; totals are computed in SQL over the whole set.

Table 6.2 Database migrations

No.	Change
0001	Initial schema
0002	Roles and sign-in attempt tracking
0003	Tenant indexes on business_id with sort columns
0004	Event author (user_id) on events
0005	Source rule and event on notifications
0006	Inbound channels and review queue
0007	Claim time on events for the sweep
0008	Payments ledger and due dates
0009	Payment modes and references

The AI layer is reached through a single complete(prompt, system, image) function. Model names are configurable because pinned names are retired from time to time, and the tests clear every key so that the suite never calls a provider.

6.5 Key Screens / System Demonstration

The screenshots in this section show the main screens of the working system.

[Insert screenshot here]

Sign-in page

Fig. 6.2 Sign-in page

[Insert screenshot here]

Dashboard with KPI cards, revenue chart and business-health score

Fig. 6.3 Dashboard with KPI cards, revenue chart and business-health score

[Insert screenshot here]

Smart Input: a note typed in Kannada or Hindi

Fig. 6.4 Smart Input: a note typed in Kannada or Hindi

[Insert screenshot here]

Smart Input: confirmation screen with matched party and products

Fig. 6.5 Smart Input: confirmation screen with matched party and products

[Insert screenshot here]

Smart Input: photo of a handwritten order slip and the resulting draft

Fig. 6.6 Smart Input: photo of a handwritten order slip and the resulting draft

[Insert screenshot here]

Sales list with source badges and payment status

Fig. 6.7 Sales list with source badges and payment status

[Insert screenshot here]

Printable invoice

Fig. 6.8 Printable invoice

[Insert screenshot here]

Record payment dialog with payment modes

Fig. 6.9 Record payment dialog with payment modes

[Insert screenshot here]

Products page with low-stock indicators

Fig. 6.10 Products page with low-stock indicators

[Insert screenshot here]

Inbox with orders received by email and Telegram

Fig. 6.11 Inbox with orders received by email and Telegram

[Insert screenshot here]

Telegram chat sending an order to the SmartSME bot

Fig. 6.12 Telegram chat sending an order to the SmartSME bot

[Insert screenshot here]

Workflow rules page with execution history

Fig. 6.13 Workflow rules page with execution history

[Insert screenshot here]

Event bus page showing events and their authors

Fig. 6.14 Event bus page showing events and their authors

[Insert screenshot here]

Reports page with charts and PDF/CSV download

Fig. 6.15 Reports page with charts and PDF/CSV download

[Insert screenshot here]

Team page: inviting a member and assigning a role

Fig. 6.16 Team page: inviting a member and assigning a role

6.6 Integration and Deployment

The frontend and backend are deployed independently (Fig. 6.17). The single-page application is built into static files and served from Vercel, which rewrites /api/* to the backend so the session cookie stays first-party. The API runs under uvicorn on an application host (Render was used) and connects to a managed PostgreSQL database (Neon was used). Migrations are applied by hand with alembic upgrade head, not at start-up.

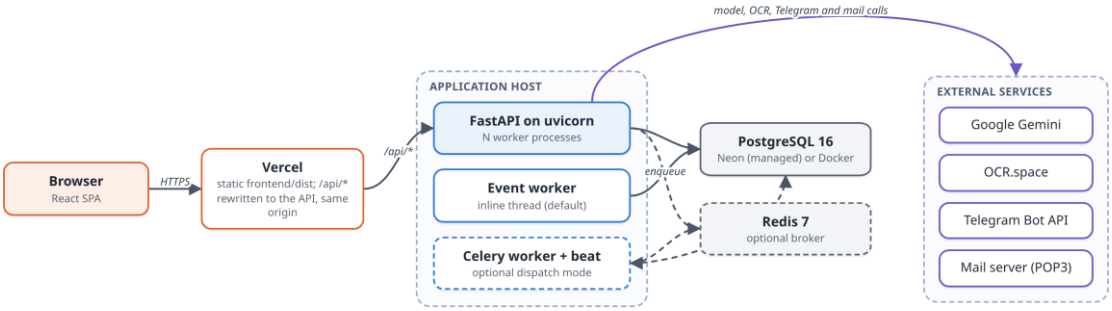


Fig. 6.17 Deployment diagram

6.7 Challenges Encountered and Solutions

Table 6.3 Challenges and solutions

Challenge	Solution
Model sometimes gave different drafts for the same note	Temperature fixed at zero: 5 of 5 notes became consistent, accuracy unchanged
Crossed-out items on slips were billed	Vision model told to ignore struck-out text; OCR-only drafts labelled
Rate-limited calls silently fell back to the regex parser during evaluation	Harness counts and reports results produced by another engine
Expense categories differed between engines	One fixed category vocabulary for both; exact match rose from 90.0% to 97.1%
Kannada suffix splitting invented customers (ಬಾಡಿಗೆ read as "to ಬಾಡಿ")	Vocabulary applied before suffixes are split
"Ram" matched "Ramesh"	Whole-word comparison instead of substring matching
An emailed purchase reduced stock	Direction taken from the party type; regression test added
Connection pool exhausted under load	Pool raised to 20 + 40 with a 10 s timeout; multiple workers
Dashboard took 1.40 s per request	Rewritten as SQL aggregates: 0.24 s
Upload requests failed because a JSON header was set on form data	API client fixed and covered by frontend tests
"ragi atta" matched the shop's wheat flour	A word before the known concept must appear in the product name

CHAPTER 7

TESTING, RESULTS AND DISCUSSION

7.1 Testing Strategy

Testing was done at four levels. Unit tests check pure logic such as totals, date parsing, matching and health scoring without a database. API tests call every route the way the frontend does and check responses and event effects. Integration tests drive whole flows: a raw email or Telegram update in, a recorded sale with moved stock out. Separately, an evaluation harness measures extraction accuracy against gold labels, and a Locust script measures performance under load. Accuracy is kept out of the test suite on purpose, because it needs network calls and is not deterministic.

7.2 Test Environment and Test Data

The API tests create a scratch database called smartsme_test, apply all migrations to it and sign up a fresh business for each run, so development data is never touched. Every AI key is cleared, so the suite makes no network calls. The evaluation data sets are listed in Table 7.1.

Table 7.1 Evaluation data sets

Data set	Size	Description
Typed notes	70	English, Hindi (Latin and Devanagari) and Kannada (Latin and Kannada script); synthetic, written by the team
Order slips	12	Real photographs of handwritten slips; three contain items struck out and rewritten
Out-of-vocabulary notes	24	Hindi and Kannada notes with unfamiliar verbs (12) and products missing from the lexicon (12)
Catalogue	17 rows	Fixed shop of 7 parties and 10 products (16 products for the out-of-vocabulary set)

7.3 Test Cases and Test Results

The backend suite has 569 automated tests in 19 layers (Fig. 7.1). Table 7.2 lists representative test cases drawn from the suite.

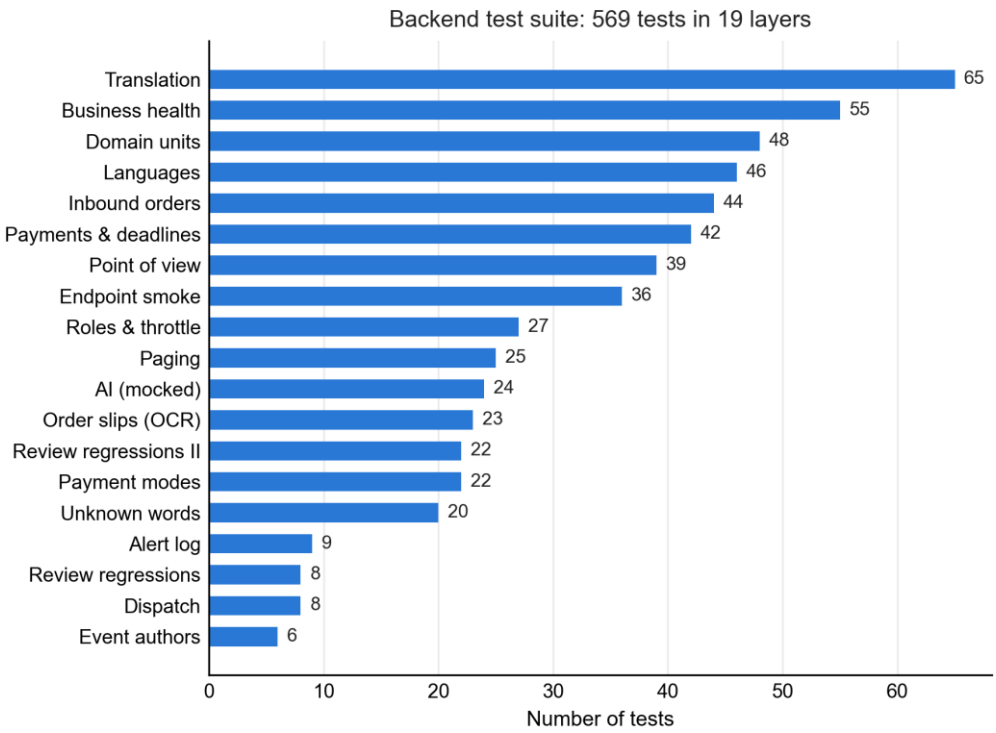


Fig. 7.1 Automated tests by layer

Table 7.2 Representative test cases

ID	Test case	Expected result	Result
TC01	Record a sale through the API	201; stock reduced before the response returns	Pass
TC02	Cancel a sale	Stock and party balance restored	Pass
TC03	Same note in five language variants	Same event, party and items each time	Pass
TC04	ಬಾಡಿಗೆ ೮೦೦೦ ಖರ್ಚು (rent expense)	Expense of 8000, category Rent; no customer invented	Pass
TC05	Model returns malformed JSON or raises	Falls back to the offline parser	Pass
TC06	Settle all dues of one party	One ledger row per document, each dated	Pass
TC07	Employee tries to delete a product	403 Forbidden	Pass
TC08	Tenant requests another tenant's sale	404 Not Found	Pass
TC09	Five failed sign-ins for an owner	Account locked; lock doubles on repeat	Pass
TC10	Celery delivers one event twice	Applied once; second delivery skipped	Pass
TC11	Emailed purchase accepted	Recorded as a purchase; stock increases	Pass
TC12	Two users accept one inbound order	One succeeds, the other gets 409	Pass
TC13	Card payment with a full card number	Refused; only last four digits allowed	Pass
TC14	Page size 1 on a list endpoint	Totals identical to the full set	Pass
TC15	"ragi atta" against wheat flour only	No match; line left for the user	Pass

[Insert screenshot here]

Terminal output of the pytest run showing the per-layer summary

Fig. 7.2 Terminal output of the pytest run showing the per-layer summary

7.4 Functional and Non-Functional Testing

Functional requirements FR1 to FR15 are covered by the API smoke tests, which call each route, and by the inbound, payment and workflow tests. Non-functional requirements were verified as shown in Table 7.3.

Table 7.3 Verification of non-functional requirements

Attribute	Test	Outcome
Consistency	Duplicate delivery, retries, stranded-event sweep	Effects applied exactly once
Reliability	AI provider faked to fail in many ways	Always degrades to the offline parser
Security	Role matrix tested from both sides; throttling; invite reuse	All refusals enforced
Privacy	Payment-mode rules; keys cleared in tests	No full card numbers stored; no network calls in tests
Performance	Locust, 200 users	106.3 req/s, p95 150 ms, 0% failures
Maintainability	Ruff lint and format; migrations smoke-tested	Clean

7.5 Performance / Model Evaluation

7.5.1 Typed Notes

Table 7.4 Extraction results on typed notes (n = 70); F1 in %

Engine	Exact	Type	Party	Items	Amount	Category
Offline parser	87.1%	100.0	94.8	93.7	95.2	100.0
Gemini	100.0%	100.0	100.0	100.0	100.0	100.0

Table 7.5 Exact match by language variant

Variant	n	Offline parser	Gemini
English	18	66.7%	100.0%
Hindi, Latin letters	18	83.3%	100.0%
Hindi, Devanagari	11	100.0%	100.0%
Kannada, Latin letters	12	100.0%	100.0%
Kannada script	11	100.0%	100.0%

Both engines were measured on the final code on 10 November 2026; eight notes met a temporary provider error on the model's first pass and were run again. In September the model scored 97.1% (Table 6.3); the two notes it missed then are now read in full. The

offline parser loses ground on messy English and Hinglish notes (no verb, no capitals, typing errors, two items on a line).

7.5.2 Handwritten Order Slips

Table 7.6 Results on real handwritten slips (n = 12)

Engine	Exact	Party F1	Phone F1	Items F1	Clean	Corrected
Vision model	100.0%	100.0	100.0	100.0	9 / 9	3 / 3
OCR.space	75.0%	100.0	100.0	91.7	9 / 9	0 / 3

Both engines read the handwriting itself correctly. The whole difference lies in the three slips where an item was struck out and rewritten: a strikethrough is not a character, so character OCR cannot see it. Fig. 7.3 compares the engines.

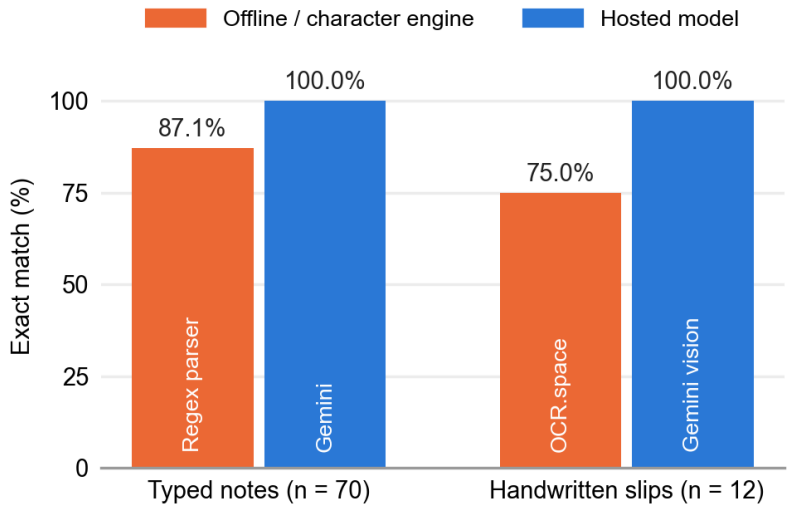


Fig. 7.3 Exact-match accuracy on typed notes and handwritten slips

7.5.3 Consistency and Latency

Five notes were run three times each. At the model's default temperature three of five gave the same draft every time; with temperature zero all five did, and accuracy was unchanged. The median time from a typed note to a draft through the model was 2.2 s; a photograph took 1.4 s with the vision model and 1.9 s with OCR.space.

7.5.4 Out-of-Vocabulary Notes

A second set of 24 notes tested words the system was not built for (Table 7.7, Fig. 7.4). Both engines were run on the final code. One note met a temporary provider error on the model's first pass and was run again, so every Gemini result is the model's own answer.

Table 7.7 Out-of-vocabulary notes, fully correct drafts

Engine	Unusual grammar	Unlisted products	Overall
Offline parser	4 / 12	3 / 12	29.2% (7 / 24)
Gemini	11 / 12	9 / 12	83.3% (20 / 24)

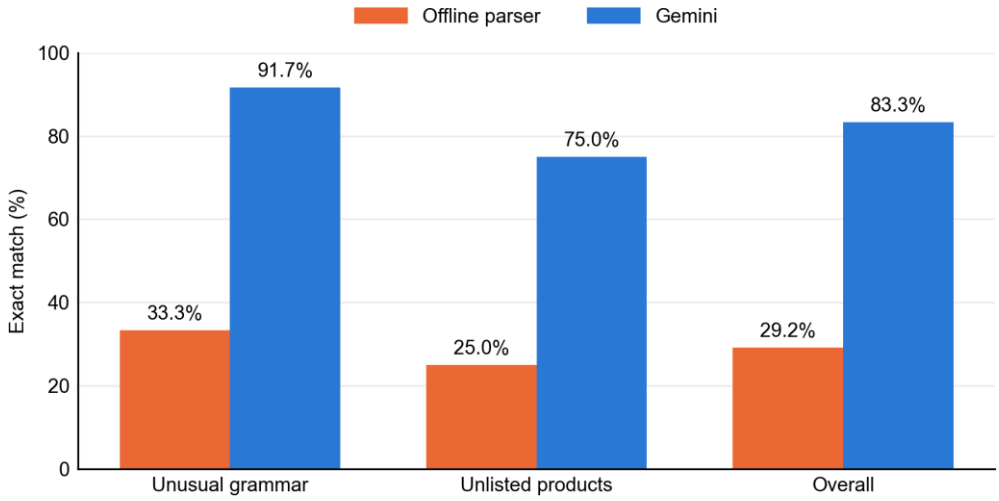


Fig. 7.4 Results on out-of-vocabulary notes

The model handled unfamiliar grammar and unlisted products far better than the offline parser. Its four remaining errors all occurred when matching names to the catalogue: two English supplier names written in Kannada script, and two product words (one Hindi, one Kannada) that matched no product.

7.5.5 Throughput Under Load

Two hundred simulated users, with about eight reads per write, were run against a business with 4,000 sales on a 16-core laptop (Table 7.8, Fig. 7.5). Each fix exposed the next limit.

Table 7.8 Load test with 200 concurrent users

Step	Requests	Failed	Req/s	Median (ms)	p95 (ms)
Baseline	129	19.4%	3.5	2,200	33,000
Larger connection pool	430	19.3%	9.8	6,500	31,000
+ 4 uvicorn workers	994	5.4%	16.8	3,500	19,000
+ SQL aggregates	6,301	0%	106.3	33	150

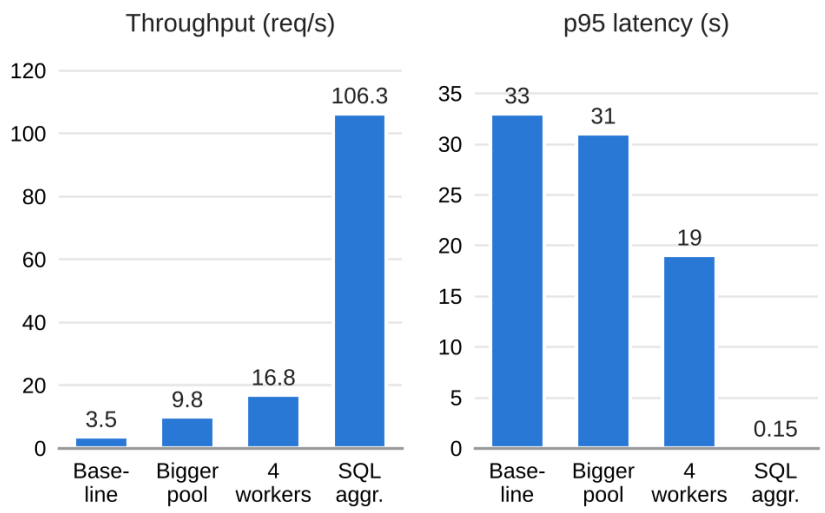


Fig. 7.5 Throughput and 95th-percentile latency after each optimisation

Adding tenant indexes reduced the sales-list query on 200,000 sales across 50 businesses from 10.50 ms (parallel sequential scan) to 0.067 ms (index scan). Rewriting the dashboard as SQL aggregates cut it from 1.40 s to 0.24 s per request.

[Insert screenshot here]

Locust run summary or evaluation harness output

Fig. 7.6 Locust run summary or evaluation harness output

7.6 Results and Observations

- On well-formed notes the offline parser comes close to the model, so the system remains useful with no external service.
- On unfamiliar phrasing the offline parser drops to 29.2% while the model reaches 83.3%, and the model's errors were all in matching rather than understanding.
- For photographs, the choice of engine matters mainly for slips with corrections, which were a quarter of a real sample.

- Throughput was limited by configuration and query design, not by the monolithic architecture.
- The evaluation found four defects that hundreds of passing tests had missed, which justified keeping accuracy measurement separate from testing.

7.7 Comparison with Existing / Baseline Approach

Two baselines are relevant. Within the system, the offline parser is the baseline for the model (87.1% against 100.0% exact match on typed notes) and character OCR is the baseline for the vision model (75.0% against 100% on slips). Against form-based software, SmartSME adds capture from chat, email and photographs, regional-language input and automatic stock and balance updates (Table 2.2 in Chapter 2), while keeping ordinary forms for users who prefer them. A timed comparison of entry speed against form-based software was not carried out and is left for the field trial.

7.8 Achievement of Project Objectives

Table 7.9 Achievement of objectives

Objective	Status	Evidence
1. Intelligent input ingestion	Achieved	Notes in 3 languages and 5 variants; slips read by vision model and OCR; Table 7.4, Table 7.6, Table 7.7
2. Asynchronous event bus	Achieved	Outbox with claim, retry, dead-letter; inline and Celery modes; dispatch tests
3. Modular ERP core	Achieved	Sales, purchases, payments, inventory, expenses, reports over a tenant-scoped schema
4. Configurable automation	Achieved	WHEN/THEN rules with six operators; executions log; alerts
5. Dashboard and evaluation	Achieved	KPIs, health score, accuracy and load evaluation reported in this chapter

7.9 Business Model Validation and User Feedback

A formal user study with shop owners has not yet been conducted. The system was demonstrated to the project guide and peers, and feedback from these sessions shaped features such as the source badges and the confirmation screen. Validation of the business model with real shops (willingness to pay, time saved per order, adoption) is planned for the field trial described in Chapter 8. Feedback collected by the team can be recorded in Table 7.10.

Table 7.10 User feedback log (to be completed by the team)

Date	Participant	Feedback and action taken

7.10 Business, Social and Environmental Impact

- Business: less time spent re-entering orders, dues tracked against due dates, and stock alerts that reduce lost sales.
- Social: staff can work in Kannada or Hindi, and employees can record sales without the power to delete records, which reduces disputes.
- Environmental: better visibility of slow-moving stock can reduce expired perishables in grocery shops.

These impacts are expected effects derived from the system's features; they have not yet been measured in operating shops.

7.11 Contribution to Selected SDG Targets

Table 7.11 Contribution to selected SDG targets

Target	Feature that contributes	Evidence available now
8.3	Digital capture of informal orders; invoices; ledgers	Working system; accuracy results in Section 7.5
9.3	Dated payment ledger; on-time collection measure	Payments and health tests; dashboard
12.3	Low-stock alerts; dead-stock share	Workflow rules; inventory part of the health score

7.12 Measurement of SDG Outcomes and Impact Indicators

The indicators defined in Chapter 4 can be computed from the system's own tables. On the demonstration business they are produced by the dashboard and reports. Real outcome values require data from operating shops over several months, so baseline and follow-up measurements will be taken during the planned field trial. Table 7.12 lists each indicator and its measurement plan.

Table 7.12 Measurement plan for SDG indicators

Indicator	Measured from	Status
Share of orders recorded digitally	Documents by source vs shop's order count	Planned (field trial)
On-time collection rate	Payments ledger and due dates	Implemented in health score
Stock-out rate of fast movers	Sales-weighted availability	Implemented in health score
Dead-stock share	Unsold stock value over 90 days	Implemented in health score
Health score trend	Monthly dashboard snapshots	Planned (field trial)

7.13 Limitations and Threats to Validity

- Internal validity: the typed notes were written by the team that wrote the parser, which favours the parser. The first fifty notes scored 100% offline before messier notes were added.
- External validity: the slips come from one writer in neat block capitals and good light; results may differ for real customers.
- Construct validity: exact match treats every field equally, although a wrong customer matters more than a wrong date.
- Reliability of labels: each gold label was written once, by one person; no inter-annotator agreement is available.
- A perfect model score on the typed notes is the ceiling of that set, not of the model; the out-of-vocabulary set is small (24 notes) and deliberately hard.
- Latency was measured on single calls; behaviour of the AI path under many simultaneous users is unknown.

CHAPTER 8
CONCLUSION AND FUTURE ENHANCEMENTS

8.1 Summary of the Project

SmartSME is a web-based business management platform for small and medium enterprises that take orders informally. It accepts typed notes in English, Hindi and Kannada, photographs of handwritten slips, and orders sent by email or Telegram, and turns each into a draft that a person confirms before it is recorded. Confirmed records and their events are stored together in a PostgreSQL outbox, from which a rule engine updates stock, balances and alerts. A dashboard summarises the business with a health score built from a dated payment ledger.

8.2 Conclusions

- Informal orders can be captured reliably enough to be useful when every draft is confirmed by a person: 87.1% of typed notes were fully correct offline and 100.0% with the language model.
- A vision model is clearly better than character OCR for handwritten slips that contain corrections, which were common in a real sample.
- A transactional outbox in PostgreSQL gives the consistency of an event-driven system without a separate broker.
- A modular monolith on modest hardware reached 106.3 requests per second for 200 users once configuration and queries were tuned.
- Multilingual support is strongest for vocabulary the system knows; unfamiliar words remain the main source of error, especially without a model.

8.3 Achievement of Project Objectives

All five objectives set in Chapter 1 were achieved, as shown in Table 7.9. Input ingestion, the event bus, the modular core, configurable automation and the dashboard are implemented, tested and evaluated, with the limitations noted in Section 7.13.

8.4 Key Technical and Business Contributions

Table 8.1 Key contributions

Type	Contribution
Technical	Confirmation-first Smart Input shared by four entry points
Technical	Cross-script matching with a retail lexicon, transliteration and a qualifier rule
Technical	Transactional outbox with atomic claims, retries, dead-lettering and optional Celery dispatch
Technical	Ledger-based business-health score with documented formulas and 55 tests
Technical	Evaluation harness that reports accuracy, consistency, latency and fallback counts
Business	Freemium model for small shops with low break-even, and SDG indicators drawn from the system's own data

8.5 Future Scope and Enhancements

Fig. 8.1 outlines the planned enhancements. The most important next step is a pilot in working shops, with notes and slips written by real customers and labelled by two annotators, to measure accuracy, time saved and adoption. Further work includes a lexicon that learns from user corrections, voice-note input, a WhatsApp channel, a small on-device language model for the offline path, a seasonal baseline for the health score, GST e-invoicing, and consent-based credit assessment from the payment ledger in line with the DPDP Act [25].

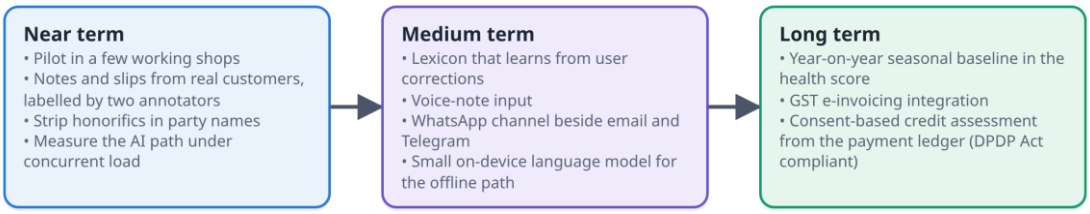


Fig. 8.1 Roadmap of future enhancements

REFERENCES

[1] Press Information Bureau, Ministry of Micro, Small and Medium Enterprises, Contribution of MSMEs to the GDP, Press Release, New Delhi, 22 July 2024.

[2] National Statistics Office, Ministry of Statistics and Programme Implementation, Annual Survey of Unincorporated Sector Enterprises (ASUSE) 2023-24, Press Note, New Delhi, 29 January 2025.

[3] P. Vashisht, MSME Digitalisation in India: Current Status and Challenges, New Delhi, Research and Information System for Developing Countries (RIS), 2025.

[4] United Nations General Assembly, Transforming Our World: The 2030 Agenda for Sustainable Development, Resolution A/RES/70/1, New York, 2015.

[5] M. Al-Mashari, A. Al-Mudimigh and M. Zairi, Enterprise resource planning: A taxonomy of critical factors, European Journal of Operational Research, Vol 146, No 2, 2003, pp 352-364.

[6] M. Haddara and O. Zach, ERP systems in SMEs: A literature review, Proc of 44th Hawaii International Conference on System Sciences (HICSS), Jan 2011, pp 1-10.

[7] G. Hohpe and B. Woolf, Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions, Boston, MA, Addison-Wesley, 1 Edition, 2003.

[8] G. Mühl, L. Fiege and P. Pietzuch, Distributed Event-Based Systems, Berlin, Springer, 1 Edition, 2006.

[9] P. Helland, Life beyond distributed transactions: An apostate's opinion, Proc of 3rd Biennial Conference on Innovative Data Systems Research (CIDR), Asilomar, CA, Jan 2007.

[10] M. Fowler, Event Sourcing, martinfowler.com, December 2005. Available: https://martinfowler.com/eaaDev/EventSourcing.html

[11] C. Richardson, Microservices Patterns: With Examples in Java, Shelter Island, N.Y., Manning, 1 Edition, 2018.

[12] M. Kleppmann, Designing Data-Intensive Applications, Sebastopol, CA, O'Reilly Media, 1 Edition, 2017.

[13] U. Dayal et al., The HiPAC project: Combining active databases and timing constraints, ACM SIGMOD Record, Vol 17, No 1, March 1988, pp 51-70.

[14] C. L. Forgy, Rete: A fast algorithm for the many pattern/many object pattern match problem, Artificial Intelligence, Vol 19, No 1, 1982, pp 17-37.

[15] R. Smith, An overview of the Tesseract OCR engine, Proc of 9th International Conference on Document Analysis and Recognition (ICDAR), Vol 2, 2007, pp 629-633.

[16] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei and M. Zhou, LayoutLM: Pre-training of text and layout for document image understanding, Proc of 26th ACM SIGKDD

International Conference on Knowledge Discovery and Data Mining, 2020, pp 1192-1200.

[17] G. Kim et al., OCR-free document understanding transformer, Computer Vision - ECCV 2022, Lecture Notes in Computer Science, Vol 13688, Springer, 2022.

[18] T. Brown et al., Language models are few-shot learners, Advances in Neural Information Processing Systems, Vol 33, 2020, pp 1877-1901.

[19] Gemini Team, Google, Gemini: A family of highly capable multimodal models, arXiv preprint arXiv:2312.11805, 2023.

[20] Z. Ji et al., Survey of hallucination in natural language generation, ACM Computing Surveys, Vol 55, No 12, 2023, pp 1-38.

[21] K. Bali, J. Sharma, M. Choudhury and Y. Vyas, "I am borrowing ya mixing?" An analysis of English-Hindi code mixing in Facebook, Proc of First Workshop on Computational Approaches to Code Switching, Oct 2014, pp 116-126.

[22] S. Khanuja, S. Dandapat, A. Srinivasan, S. Sitaram and M. Choudhury, GLUECoS: An evaluation benchmark for code-switched NLP, Proc of 58th Annual Meeting of the Association for Computational Linguistics, July 2020, pp 3575-3585.

[23] indic-transliteration: Transliteration tools to convert text in one Indic script encoding to another, Python Package Index, version 2.3. Available: https://pypi.org/project/indic-transliteration/

[24] E. I. Altman, Financial ratios, discriminant analysis and the prediction of corporate bankruptcy, The Journal of Finance, Vol 23, No 4, Sep 1968, pp 589-609.

[25] Government of India, The Digital Personal Data Protection Act, 2023, Act No. 22 of 2023, Ministry of Law and Justice, New Delhi, 2023.

[26] K. Moriarty, B. Kaliski and A. Rusch, PKCS #5: Password-Based Cryptography Specification Version 2.1, RFC 8018, Internet Engineering Task Force, Jan 2017.

[27] M. Jones, J. Bradley and N. Sakimura, JSON Web Token (JWT), RFC 7519, Internet Engineering Task Force, May 2015.

[28] A. Osterwalder and Y. Pigneur, Business Model Generation, Hoboken, N.J., John Wiley & Sons, 1 Edition, 2010.

[29] R. S. Sandhu, E. J. Coyne, H. L. Feinstein and C. E. Youman, Role-based access control models, IEEE Computer, Vol 29, No 2, Feb 1996, pp 38-47.

[30] N. Chinchor, MUC-4 evaluation metrics, Proc of Fourth Message Understanding Conference (MUC-4), 1992, pp 22-29.

PUBLICATIONS

Published Journal / Conference / Patent

[1] [Author names], "SmartSME: A Confirmation-First, Event-Driven Platform for Informal Order Capture and Bookkeeping in Small and Medium Enterprises," [Conference or journal name], [Status: submitted / accepted / published], [Year].

[Attach the acceptance letter, publication certificate or patent filing receipt after this page, if available.]